

Damage the Operation-Precedence Graph Cannot Cover: Spatiotemporal Reservation Closures in Conflict-Free Flexible Job Shops

QI YU, Southeast University, PR China

WEI LI*, Southeast University, PR China

In flexible job shop scheduling where operation precedence captures the processing order while time-window reservations record corridor occupancies, the off-the-shelf way of delineating the rescheduling scope after a disturbance is still to identify the affected operations first and then to reschedule them together with their successors. Yet when a corridor becomes impassable, or slower to traverse, over some time window, no operation becomes unexecutable as a consequence: that way of proceeding returns an empty scope and declares that no rescheduling is needed. Meanwhile the corridor occupancies that fall inside the disturbed interval have already been invalidated, the original schedule is infeasible, and the induced waiting propagates to downstream vehicles along yielding relations. The gap lies not in detection but in the fact that the operation-precedence graph does not contain the yielding relations among corridor occupancies.

This paper defines the rescheduling scope on the corridor occupancies and their dependencies, and gives the spatiotemporal reservation closure (STRC), called the reservation impact set in the body: starting from the corridor reservations whose occupancy interval overlaps the disturbed window, the transitive closure is taken along four kinds of edges—yielding on the same corridor, the next occupancy of the same vehicle, and the occupancies of the next operation of the same job and of the same machine; occupancies inside the set may be rewritten, while those outside it keep the original plan. Once the dependency edges are given, whether an occupancy belongs to the set is uniquely determined and does not vary with how the rerouting algorithm is implemented. Recovery proceeds at two levels: first reroute the original vehicle inside the set; if that remains infeasible, enlarge the rewritable scope in a prescribed order.

On 5 instances $\times$ 10 random seeds, that is 50 pairs in total, the scope delineated on the operation-precedence graph under corridor blockage is empty on 50/50 of them while the original schedule is indeed infeasible; holding the rerouting procedure fixed and replacing only the object the boundary is drawn on, the rate at which feasibility is restored moves from 0/50 to 50/50; no dependency edge leads from inside the set to outside it, and the corridor, interval and vehicle of every occupancy outside the set are unchanged entry by entry, at 50/50 and 50/50 respectively. Replacing the rewritable set by “every occupancy not yet finished at the decision time” and leaving everything else untouched, runtime and makespan barely move, whereas the rewrite fraction rises from 0.540 to 0.604 (per cell 31/18/1, two-sided Wilcoxon signed-rank $p = 8.8 \times 10^{-7}$). Against warm-started global rescheduling that likewise never rewrites finished occupancies, bounded repair is two to three orders of magnitude faster, but its makespan is worse at every budget level and no crossover appears as the budget is relaxed; on makespan it is no substitute.

The larger effect comes from the object the boundary is drawn on: the restored feasibility reported above follows from replacing that object alone, the rerouting procedure being untouched. The smaller effect comes from the impact set itself, namely fewer rewrites; but the set already covers about 92% of “every occupancy not yet finished”, so as the two approach each other this benefit tends to

*corresponding author

Authors' Contact Information: Qi Yu, 230240012@seu.edu.cn, School of Cyber Science and Engineering, Key Laboratory of Computer Network and Information Integration, Southeast University, Nanjing, Jiangsu, PR China; Wei Li, xchlw@seu.edu.cn, School of Computer Science and Engineering, Key Laboratory of Computer Network and Information Integration, Southeast University, Nanjing, Jiangsu, PR China.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

vanish, and no minimality is claimed. Rerouting inside the set, keeping the plan outside it, and enlarging the scope after a failure have all appeared in prior work; what this paper claims is not a new repair strategy but the object the boundary is drawn on.

CCS Concepts: • **Theory of computation** → **Scheduling algorithms**; • **Applied computing** → *Industry and manufacturing*; • **Computing methodologies** → *Planning and scheduling*.

Additional Key Words and Phrases: flexible job shop scheduling, conflict-free AGV routing, spatiotemporal reservation table, spatiotemporal reservation closure, corridor blockage

ACM Reference Format:

Qi Yu and Wei Li. 2026. Damage the Operation-Precedence Graph Cannot Cover: Spatiotemporal Reservation Closures in Conflict-Free Flexible Job Shops. 1, 1 (September 2026), 45 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

1.1 Background and motivation

Flexible job shop scheduling with transportation has long rested on one convenient assumption: the time it takes to move a job between two machines is a constant, to be read off a matrix indexed by pairs of locations. The assumption presumes that vehicles never contend for the same corridor, so that travel time is fixed by the distance between two points rather than being an outcome of routing decisions. As soon as a finite fleet shares a sparse corridor network and reserves the contended corridors by time windows, travel time stops being given data and becomes the outcome of two decisions—which path a vehicle takes, and who reserved that corridor first. One vehicle yielding delays the arrival of another, which in turn delays the start of a downstream operation and of its transport occupancy. A feasible schedule must therefore fix the operation assignment, the processing sequence and a reservation table recording corridor occupancies all at once; and a disturbance can not only break operations but also invalidate corridor occupancies alone, without changing any operation assignment.

Work that completes the schedule once before release, deciding machine assignment and conflict-free paths together, has already shown the following: when an operation has several eligible machines, contention for corridors changes which machine is preferable—if the corridor leading to the faster machine is occupied, a slower but more accessible machine may be the better choice. Evaluating a candidate schedule therefore has to account for waiting through time-window routing, and the search has to query the reservation table for conflicts, instead of continuing to use constant travel times [9, 17, 20]. What the literature above answers is how to produce a high-quality conflict-free schedule when information is complete and work has not yet started. For the same shop, once execution has reached time t_{now} and a disturbance has occurred, the question becomes a different one: which corridor occupancies does the damage fall on, how far must the rewriting reach before feasibility is restored, and can all of this be done within milliseconds to hundreds of milliseconds without restarting a population search.

1.2 The gap the operation-precedence graph cannot cover

In research on dynamic scheduling of flexible job shops with transportation, the rescheduling scope is usually delineated along the operation-precedence graph: starting from the operations that can no longer run as planned because of a machine breakdown, one adds the downstream operations of the same job and the operations sequenced after them on the same machine, or else stops after a bounded number θ of steps from the starting set on the graph, and then reschedules only the operations inside the scope. Two dynamic-scheduling studies that include AGV transportation and take machine breakdown or order insertion as the disturbance draw the boundary in exactly this way—their recovery

scope is, respectively, “the operations not yet processed after the breakdown” [24] and “the operations and delivery tasks after they have been sorted, by how the disturbance affects them, into retained, continued and reconstructed” [19] (the latter is a batch-flow hybrid flow shop whose disturbances include vehicle breakdown, but whose model has no reservation table). The practice traces back to affected-operations rescheduling (AOR) [19, 24]. Later on we set up a comparison arm that delineates the scope on the operation-precedence graph, in order to reproduce this still-current practice, not to improve AOR itself (Section 2.1).

This way of delineating the scope is often reasonable when the disturbance is itself an operation-level event—for instance when a machine breakdown makes several operations impossible under their current assignment. When a schedule records corridor occupancies by time windows, however, there is another class of disturbance: it renders a corridor impassable over some interval without changing any operation assignment. The typical example is corridor blockage: passage through the corridor is forbidden during $[t_a, t_b)$. A blockage modifies neither the machine assignment nor the processing sequence, so no operation on the operation-precedence graph is directly touched, the rescheduling scope delineated by that method is empty, and on that basis the disturbance is declared not to require rescheduling. The corridor occupancies that must nevertheless be recovered therefore lie beyond the reach of the operation-precedence graph: the occupancies of the original plan that overlap the blockage interval already conflict with the prohibition and are no longer feasible; a vehicle consequently waits or detours, which in turn blocks downstream vehicles along yielding relations, and the invalidation propagates through the reservation table.

“Beyond the reach” here means that the operation-precedence graph contains, by definition, no yielding relations among corridor occupancies and therefore cannot express how an invalidated occupancy propagates from vehicle to vehicle; it does not mean that a sensor missed a detection or that an algorithm happened not to look. If travel times were still taken to be a constant matrix indexed by pairs of locations, corridors would not be controllable resources, no reservation table would exist, and damage of this kind could not even be written into the model. The gap can therefore hold only in a setting where corridor occupancies are recorded by time windows; this is not a deliberate narrowing of scope but a prerequisite for stating the problem clearly.

Accordingly we distinguish two classes of disturbance.

- **Class A** (acting on the operation-precedence graph): robotic-arm breakdown, marked deviation in processing time, order insertion, and *vehicle breakdown*. Disturbances of this class make some operations impossible to run as planned, so the set of starting operations is non-empty when the rescheduling scope is delineated; both the operation-precedence graph and the reservation impact set return a non-empty scope, which is why this class is not our primary vehicle for exposing a misplaced boundary. The reason vehicle breakdown belongs to Class A is given in Section 3.3: although it changes no operation assignment, it makes the operations that the vehicle was carrying unexecutable, so through the correspondence “vehicle to the operations it carries” the operation-precedence graph still has a non-empty set of starting operations.
- **Class B** (acting on the reservation table only): corridor blockage, corridor slowdown, and temporary withdrawal of a local right of way, among others. Disturbances of this class make no operation unexecutable, and the set of starting operations on the operation-precedence graph is empty; the corridor occupancies overlapping the disturbed interval are non-empty, and so is the reservation impact set they bring into the rescheduling scope.

Class B constitutes a counterexample at the level of the problem: the issue is not that existing methods repair operations less well than they might, but that in a model which uses the operation-precedence graph to capture processing order *and* time windows to record corridor occupancies, the off-the-shelf rescheduling scope is still drawn on the

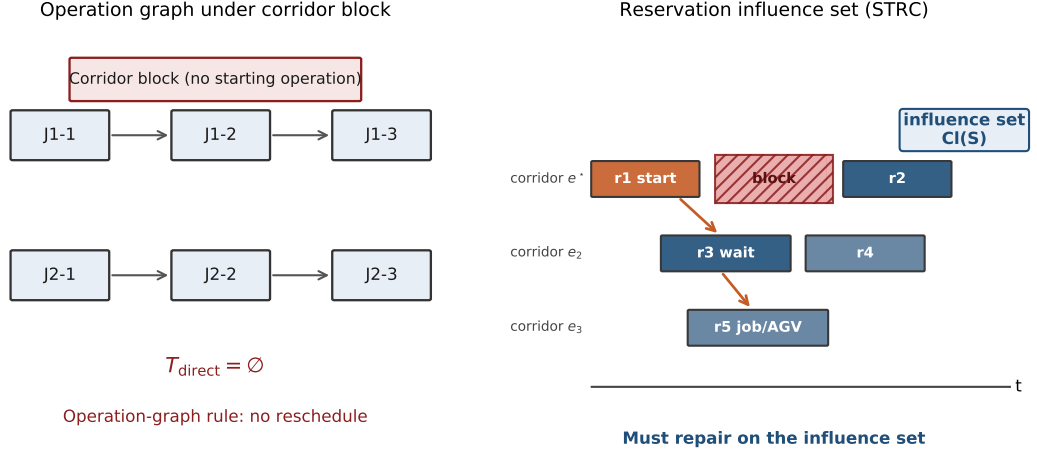


Fig. 1. Motivating illustration. Left: a corridor blockage makes no operation and no machine unable to run as planned, so the operation-precedence graph has no operation to start from. Right: under the same disturbance, the corridor occupancies overlapping the blockage interval become the starting set and grow into a reservation impact set along yielding and succession relations.

operation-precedence graph, a graph that cannot express a disturbance such as corridor blockage. The condition for this mismatch is that the model contains both families of objects yet draws the boundary using only the operation family. The two AGV-equipped dynamic-scheduling studies cited above do not exhibit the problem of “a scope drawn on the operation-precedence graph and therefore blind to corridor blockage” because their models do not consider corridor contention between vehicles (Section 2.4), which leaves the operation-precedence graph as the only object available for delineating the scope—evidence that the mismatch arises only when both families of objects are present at once, rather than being an oversight in any one piece of work. The claim of this paper is therefore confined to the setting described above and is not that every piece of local-rescheduling literature suffers from this mismatch (Section 2.5). Figure 1 contrasts the gap: on the left, the operation-precedence graph has no operation to start from and the disturbance is judged not to require rescheduling; on the right, the occupancies of the reservation table that overlap the blockage interval become the starting set and grow into a reservation impact set along yielding and succession relations.

1.3 Claims of this paper

This paper makes three claims.

First, when a flexible job shop uses time-window reservations to guarantee conflict-free vehicle movement, the post-disturbance rescheduling scope should be defined on the reservation dependency graph formed by corridor occupancies and their yielding relations, and not on the operation-precedence graph alone.

Second, the object the boundary is drawn on is the spatiotemporal reservation closure (STRC, Spatiotemporal Reservation Closure), that is, the reservation impact set: starting from the corridor occupancies that overlap the disturbed window and are therefore invalidated, every corridor occupancy that must directly or indirectly be voided and rewritten is brought into the rescheduling scope along yielding, the next occupancy of the same vehicle, the occupancy of the next operation of the same job, and the occupancy of the next operation on the same machine. Once these

dependency relations are given, an occupancy outside the impact set is no longer a further step from an occupancy inside it along those relations, so “which occupancies are to be rewritten” corresponds to a checkable set rather than to a search radius fixed by hand.

Third, once the impact set is in hand, the original vehicle is rerouted inside it while the original occupancies are kept outside it; if a feasible schedule is still not obtained, the rewritable scope is enlarged in the order same job, same vehicle, and finally all unfinished occupancies. For Class-A breakdowns, swapping vehicles and reassigning machines are also allowed, while the processing sequence still does not change. These steps serve to put the impact set to work; what this paper claims lies in how the impact set is delineated, not in whether these repair steps are novel.

The three claims share one position: **the reservation impact set is a measurement, not a decision**. This paper does not use “who yielded to whom” to steer a search towards different assignments or paths—that would be improving the scheduling decision; once one corridor wait is removed, some other constraint usually becomes the new bottleneck, so gains in makespan are easily cancelled. We use the same record of “who yielded to whom” to delineate which occupancies have already been damaged: with the dependency relations given, whether an occupancy belongs to the impact set depends only on whether it is reachable from the starting set along those relations, the outcome is uniquely determined, and it does not change with whichever constraint later becomes the new bottleneck.

This does not amount to claiming that the impact set already contains every occupancy that must physically be rewritten before feasibility can be restored. Which dependency edges to draw and which to leave out is a modelling choice; once the edge set is fixed, the fact that the impact set is closed outwards (no occupancy inside it has a dependency edge leading outside) is a corollary of the definition (Proposition 2), not a property proved separately. Whether the impact set is the smallest set of occupancies needed to restore feasibility is something this paper does not claim: a smallest set depends on which repair actions are permitted, and the impact set deliberately does not commit to any one repair method (Remark 2).

Relative to global rescheduling, which starts from the original schedule and searches anew within a time limit, the reservation impact set answers a different kind of question: when an executable plan must be produced very quickly, can feasibility be restored and the rewriting confined to the impact set? After the repair, the corridor, interval and vehicle of every corridor occupancy outside the impact set agree with the original plan—a sufficient condition that holds when the edge set is given and the occupancies outside the set have been successfully written into the new table as planned (Proposition 3), not an identity that holds unconditionally. Relative to global rescheduling that does not rewrite occupancies already finished at the decision time, the makespan $C_{\max}$ obtained by bounded repair is worse, and the fraction of occupancies rewritten is already of the same order on both sides; its strength is a far shorter algorithm runtime (milliseconds against seconds). Algorithm runtime and rewrite fraction should not be explained together, and Section 6.9 tests them separately: the short runtime comes from running path planning once for each transport segment inside the impact set without restarting a population search, and not from the impact set itself; as for whether the impact set rewrites fewer occupancies, that cannot be read off the global-rescheduling comparison above—the two fractions are already close—and requires a separate comparison in which the rerouting procedure is identical and the only difference is whether what may be rewritten is the impact set or every unfinished occupancy. On the 6 instance-seed cells measured, global rescheduling attains the smaller $C_{\max}$ at every search time limit, and no case was observed in which bounded repair was better instead. The role of bounded repair is therefore not to reduce makespan but to produce an executable repair within milliseconds. When the boundary is drawn on the operation-precedence graph, a corridor blockage finds no affected operation, local rescheduling is never triggered, and the original schedule stays infeasible. Starting global rescheduling on the same disturbance would handle it, but with an algorithm runtime

on the order of seconds. When feasibility must be restored at once, one has to identify the extent of the damage on the corridor occupancies first, and then choose local recovery steps that match it. What this paper solves is that task of identification and recovery, not the task of finding a rescheduling algorithm with a better makespan.

1.4 Contributions

The contributions of this paper are as follows.

- (1) **Identifying a class of damage beyond the reach of the operation-precedence graph.** When a schedule uses operation precedence to capture the processing order and time windows to record corridor occupancies, the off-the-shelf rescheduling scope is still delineated by affected operations. Disturbances such as corridor blockage and corridor slowdown change no operation assignment, the scope delineated by that method is empty, and on that basis no rescheduling is declared necessary; yet the corridor occupancies overlapping the disturbed interval have already been invalidated, the original schedule is infeasible, and the waiting propagates to downstream vehicles along yielding relations. The gap is not a missed detection but the fact that the operation-precedence graph contains no yielding relations among occupancies; were travel times still taken to be a constant matrix, corridors would not be controllable resources and damage of this kind could not even be written into the model.
- (2) **Defining the rescheduling scope as the reservation impact set.** Starting from the corridor occupancies overlapping the disturbed window, and following yielding, the next occupancy of the same vehicle, and the occupancies of the next operation of the same job and of the same machine, the records that must directly or indirectly be voided and rewritten are brought into the rescheduling scope, giving the spatiotemporal reservation closure (STRC). Once the dependency relations are given, whether an occupancy belongs to the impact set is uniquely determined, and no occupancy inside the set has a dependency edge leading outside it, so “which ones to rewrite” corresponds to a checkable set rather than a search radius fixed by hand, and does not vary with how any particular rerouting algorithm is implemented. The impact set is a measurement, not a scheduling decision: we use “who yielded to whom” to delineate damage, and not to steer a search towards different assignments or paths. We do not claim that the impact set is already the smallest set of occupancies needed to restore feasibility (a smallest set depends on which repair actions are permitted; Remark 2). *The evidence for this layer—whether the impact set stands as an object—consists of:* under corridor blockage the scope on the operation-precedence graph is empty while the original schedule is indeed infeasible, 50/50 (Section 6.2); on Class A the reservation impact set loses none of the occupancies the operation-precedence graph already included, 50/50 (Section 6.3); and the differential probe that ablates the boundary leaks 0/50. *Whether this way of drawing the boundary is worth anything in net terms belongs to another layer:* our evidence there consists of the single RA comparison (Section 6.9), namely how many fewer occupancies are rewritten relative to “rewrite every unfinished occupancy”.
- (3) **Giving a local recovery that matches this boundary, and keeping straight where each reading comes from.** Inside the impact set the original vehicle is rerouted, outside it the original occupancies are kept; if feasibility is still not attained, the rewritable scope is enlarged. These steps serve to put the impact set to work and are not themselves the methodological contribution of this paper. Under one and the same corridor network, time-window router and conflict-free checker, the comparisons show that when the boundary is drawn on the operation-precedence graph, corridor blockage is recovered on 0/50 of the cells, whereas with the boundary drawn on the reservation impact set it is recovered on 50/50; feasibility therefore comes from the object the boundary is drawn on and not from the quality of the rerouting procedure. Bounded repair restores feasibility with a runtime on the order of milliseconds,

but its makespan $C_{\max}$ is worse than that of global rescheduling, and relaxing the budget produces no crossover (12 instance–seed cells, no crossing point inside the budget range measured). Relative to the comparison that rewrites every unfinished occupancy, runtime and $C_{\max}$ barely change, and only the smaller number of rewritten occupancies can be attributed to the impact set (Section 6.9). In the cross of four disturbance types with two ways of drawing the boundary, the operation-precedence scope is empty only under corridor blockage and corridor slowdown, the reservation impact set is non-empty throughout, and under machine or vehicle breakdown it is no smaller than the operation-precedence scope.

In the same shop and on the same reservation table, integrated pre-release scheduling uses the reservation table to build a conflict-free plan [9, 17, 20]; this paper uses the reservation table at execution time to locate the extent of the damage. The two act at different moments and do not substitute for each other.

1.5 Organization

Section 2 reviews related work in four classes: how job shops delineate the rescheduling scope by operations or along the time axis, how railway rescheduling expresses the order of occupancies on an exclusive resource, how conflict-free multi-vehicle routing replans locally, and where flexible job shops with transportation draw the boundary; our own position follows from this review. Section 3 gives the corridor-reservation setting and the execution-time recovery problem: the shop, the corridor reservation table and the operation-precedence graph, the two classes of disturbance, and the conditions a recovery must satisfy. Section 4 defines the spatiotemporal reservation closure, that is the reservation impact set, and states its dependency edges and its outward closedness (Figures 2 and 3). Section 5 explains how the original vehicle is rerouted inside the impact set while the original occupancies are kept outside it, how the rewritable scope is enlarged when feasibility is still not attained, and how all of this compares with global rescheduling (Figure 4). Section 6 reports feasibility, algorithm runtime, rewrite fraction and makespan, and uses a case Gantt chart to place the statistical results on the operation time axis. Section 7 concludes and discusses limitations and future work.

2 Related work

One basic question in dynamic scheduling is: once a disturbance has occurred, on what object is the rescheduling scope drawn, that is, what is changed and what is left alone. The existing literature draws this scope on different objects—operations or the time axis, the order of occupancies on an exclusive resource, the affected vehicles, and the set of operations in a flexible job shop with transportation. Below we review, for each of these four classes, what it does and what it does not yet cover, and summarize the review in Table 2.

2.1 Local rescheduling in job shops

For operation-level events such as machine breakdown and order insertion, delineating the rescheduling scope by affected operations is still in use in dynamic scheduling with AGVs. Zhang et al. [24] treat machine breakdown and order insertion and take the recovery scope to be the operations not yet processed after the breakdown; Tang et al. [19] sort operations and delivery tasks into retained, continued and reconstructed according to how the disturbance affects them, and place complete rescheduling, partial rescheduling and right shift side by side, noting that a right shift preserves stability at the cost of optimization room. The original sentences on which the paraphrases of this and the next subsection rest are collected in Table 1. The practice traces back to affected-operations rescheduling (AOR) [1, 16]: only the operations directly or indirectly touched by the disturbance are rearranged. Drawing the boundary along

the time axis, and enlarging the scope after a failure, have likewise been studied, including match-up [2] and Local Rescheduling with incrementally widened time windows [8]. A right shift, by contrast, decides nothing about who is affected and simply pushes the unfinished part back as a whole. Rolling-horizon and decomposition methods trade off scale against quality [23, 26]; for surveys see [14, 21].

This body of work shows that delineating a scope and then rescheduling locally is not a new problem, provided that the disturbance acts on operations in the first place. Once a disturbance only makes a corridor impassable without changing any assignment, the scope delineated by that method can be empty while the original schedule is already infeasible.

2.2 The order of occupancies on an exclusive resource

“Who occupies the same exclusive resource first” has been modelled as a graph and studied systematically in railway rescheduling. The alternative graph of Mascis and Pacciarelli [12] generalizes the disjunctive graph as follows: a vertex is the occupancy of a block section by a job, fixed arcs express the order along a route, and a pair of alternative arcs expresses who passes through that section first. D’Ariano et al. [4] use the model for real-time train rescheduling, recomputing a conflict-free timetable after a disturbance while staying as close as possible to the original plan. Yielding on a shared corridor and an alternative arc express the same kind of ordering.

What differs from the present paper is the use made of it. For every point of contention the alternative graph offers two mutually exclusive arcs (either A passes first, or B does), and scheduling consists in selecting one of them: after the selection the graph must contain no positive-weight cycle for the timetable to be feasible, and the objective is to be made as good as possible—the order of occupancies is therefore a decision variable. In this paper the plan has already been produced, who passes through which corridor first has already been fixed, and the order is no longer selected; those given yielding and succession relations are used only to bring the original corridor occupancies that must be voided after a disturbance into the rescheduling scope. Furthermore, the railway model treats a block section as a machine, so that operations and occupancies live on one and the same graph and an unavailable section simply is an operation-level event; the situation of “a scope drawn on the operation-precedence graph and therefore blind to corridor blockage” does not arise. In a flexible job shop, processing and corridor occupancy are two distinct families of objects, and the off-the-shelf AOR boundary cannot make use of this order of occupancies.

2.3 Local replanning in conflict-free multi-vehicle routing

In multi-agent path finding, rescheduling only the affected agents while treating the rest as dynamic obstacles is already standard practice. Švancara et al. [18] give Replan All, Replan Single and Replan Affected for online MAPF and use online independence detection to determine the affected set; Malka et al. [11] likewise replan only for the vehicles affected by observed changes when the map is inaccurate. Conflict-free path planning treats spatiotemporal occupancy as a first-class constraint, but the origins and destinations are usually given externally [25].

This line of work has already made “change only the affected agents, keep the original plan for the rest” into a standard mechanism, and we do not claim it as a contribution. A second difference from our setting is that such work has only vehicle origins and destinations, with no job–operation–machine assignment, so “the rescheduling scope is delineated on the operation-precedence graph and corridor blockage is missed because no affected operation can be found” cannot arise. As to how the affected set is obtained, MAPF finds the affected vehicles by independence detection, which yields the output of a detection algorithm and therefore varies with the implementation and the stopping condition. The reservation impact set given here is different: once the dependency edges are given, whether

an occupancy belongs to the set depends only on whether it is reachable from the starting set along those edges, independently of which rerouting algorithm is used afterwards (Section 4.2).

2.4 Flexible job shop scheduling with transportation

Flexible job shop scheduling with transportation vehicles has been surveyed systematically [22]. Recent combinatorial-optimization work still commonly takes travel time to be a constant matrix indexed by pairs of locations, and develops constraint programming, metaheuristics and knowledge-guided search on top of it [5, 13, 15]. Under that convention vehicles do not contend for the same corridor, corridor blockage has no corresponding object in the model, and it cannot be tested.

Once corridor occupancies are recorded by time windows, processing order and corridor occupancy enter the model together. Congestion-aware dispatching can select a vehicle using an indirect measure such as local density, without changing the machine assignment [7]. Work that genuinely brings flexible assignment and conflict-free transportation together is comparatively scarce: van Os and Basten [20] formalize flexible job shop scheduling with conflict-free transportation constraints and, with a decomposition method, attain provable optimality on most benchmark instances, optimality not being guaranteed on a few because of a time limit; a two-stage scheme first fixes production and then resolves vehicle conflicts [17]; and there is also a framework in which a learning layer schedules production, a routing layer resolves conflicts, and the two are connected by limited feedback [9]. What all of these answer, however, is how to produce a feasible and good-quality conflict-free schedule before release.

To the authors' knowledge, no work has yet taken both a corridor reservation table and execution-time disturbance response into account. This sentence is a bounded one: we do not claim to have exhausted the literature, and therefore do not state it as a universal negative. What can be checked paper by paper is that in the work cited in this section the two conditions never appear together—those with a reservation table exclude execution-time breakdowns, and those that handle execution-time disturbances have no reservation table. The following two points rest on the modelling assumptions stated in the originals (the sentences are reproduced verbatim in Table 1): Zhang et al. [24] state in their modelling assumptions that collisions between AGVs are ignored; Sun et al. [17] resolve vehicle conflicts with a time-window Dijkstra search, yet state in their assumptions that vehicle charging and breakdowns are not considered and that vehicles are always available, and list equipment maintenance and breakdowns as future research.

The gap, then, is not that some existing paper drew the boundary in the wrong place, but that in the work listed above the two conditions were never in place at once: an operation-level boundary holds, often because there is no corridor reservation; and settings that do have a reservation table exclude execution-time disturbances. Once both are present, the only off-the-shelf option is to delineate the scope by affected operations, and for a corridor blockage that changes no operation assignment this returns an empty scope (50/50 as measured in Section 6.2). This is the situation the present paper addresses. It should be noted that **the counterexample itself does not depend on the gap statement above holding**: even if existing work did incorporate both conditions, the readings of Section 6.2 would still stand, and what would be affected is our novelty claim, not its correctness.

The gap assessment above and the positioning in Section 2.1 both rest on paraphrases of other people's work, and a paraphrase can distort. So that a reader need not fetch the originals to check, the sentences on which these paraphrases rest are listed in Table 1. The immediate reason for the list is that one of them failed the check: the conclusion of van Os and Basten [20] was originally paraphrased here as "a benchmark with provable optimality obtained by a decomposition method", whereas the original confines this to most instances, optimality not being guaranteed on a few because of

Table 1. Paraphrases on which the positioning depends, against the cited originals. The five paraphrases *from which the gap is drawn* are listed, each with the original sentence of the cited work, so that the reader can judge the fidelity of the paraphrase without fetching the original. Row 4 is a paraphrase already corrected: the earlier wording was stronger than the original and has now been narrowed to match it (see the body).

Our paraphrase	Cited original	Location in the original
The recovery scope of Zhang et al. [24] is the operations not yet processed after the breakdown	<i>"Rearrange the unprocessed operations of current scheme considering the limit of machine breakdown"</i>	Rescheduling procedure
Zhang et al. [24] state in their modelling assumptions that collisions between AGVs are ignored	<i>"The collision of AGVs are ignored."</i>	Modelling assumption (7)
Tang et al. [19] sort operations and delivery tasks into retained, continued and reconstructed according to how the disturbance affects them	<i>"an event-driven partial rescheduling strategy is proposed, in which the disrupted operations and delivery tasks are classified into three categories: retained, continued, and reconstructed"</i>	Abstract
van Os and Basten [20] attain provable optimality on most benchmark instances, optimality not being guaranteed on a few because of a time limit	<i>"For most benchmark instances, our LBBD-based approach finds optimal makespan values. Because of time boxing, only in a few cases, the optimality of the found solution cannot be guaranteed."</i>	Abstract
Sun et al. [17] assume that vehicles are always available and do not consider breakdowns or charging, and list equipment maintenance and breakdowns as future research	<i>"It does not consider the charging problem and fault problem of the AGVs, and AGVs are always available"; "Breakdowns and charging problems are not taken into account."; "Future research will investigate the impact of equipment maintenance and breakdowns on production scheduling"</i>	Assumption list, future work

time boxing. The direction of the deviation is worth recording—it strengthened the conclusion on behalf of the cited work, and what was strengthened happened to be the very baseline this paper compares against.

2.5 Our position

In summary, in a flexible job shop with conflict-free AGV routing, the schedule uses operation precedence to capture the processing order and time windows to record corridor occupancies. The situation identified as a gap in the previous section then arises: disturbances such as corridor blockage do not make operations unexecutable and the original assignment remains valid, yet the corridor occupancies overlapping the disturbed interval have been invalidated and the original schedule is infeasible. The comparison arm that delineates the scope on the operation-precedence graph, used later on, reproduces the practice of Section 2.1. This paper argues for moving the rescheduling scope onto the corridor occupancies together with their yielding and succession relations, which yields a reservation impact set uniquely determined by those relations.

3 The corridor-reservation setting and the execution-time recovery problem

3.1 System setting and notation

The problem setting follows the flexible job shop with conflict-free transportation constraints [20]. This section states once and for all the objects used later. A feasible schedule $\mathcal{S}^0$ is regarded as given input: how it was found by search at $t = 0$ has no bearing on how the occupancies that must be rewritten are delineated later.

3.1.1 The shop and the four groups of decisions. The routing graph of the shop $G = (V, E)$ is strongly connected. Robotic arms occupy fixed nodes $\text{loc}(m)$, and $v_0 \in V$ is the load/unload station. There are n jobs, a machine set M , and a

Table 2. Related work classified by *the object the boundary is drawn on*: on which kind of object each class of work defines “what needs to be rewritten” after a disturbance. “Boundary drawn on” is the object the rescheduling scope is attached to; “operation-precedence graph?” also answers whether that graph shares a graph with the occupancies; the bold entry in the last row is the class this paper belongs to.

Class	Representative work	Boundary drawn on	Operation-precedence graph?	Relation to this paper
Local rescheduling in job shops	AOR [1, 16]; match-up [2]; LRS [8]; dynamic scheduling with AGVs [19, 24]	Operations / time windows	Yes	Reproduced later as the operation-level boundary and the right-shift comparisons
Order of occupancies on an exclusive resource	alternative graph [12]; real-time railway rescheduling [4]	Occupancy of a block section	Yes, on the same graph as the occupancies	Yielding relations are of the same kind; railways use them to choose the order of vehicles, whereas we use the given order to delineate the occupancies that need rewriting
Conflict-free multi-vehicle routing	Online MAPF [11, 18]; LMAPF [25]	Affected vehicles	No	Rerouting inside the set and keeping the plan outside it is already standard practice
Flexible job shop with transportation	constant matrix [5, 13, 15, 22]; integrated scheduling [9, 17, 20]	Set of operations	Yes; a reservation table and execution-time disturbances never appear together	Our position

homogeneous single-load fleet $K = \{1, \dots, N_A\}$. Job j has a fixed operation chain $O_{j1} \rightarrow \dots \rightarrow O_{jn_j}$; the set of eligible machines of operation O_{ji} is $\Omega_{ji} \subseteq M$, and its processing time on machine m is t_{jim}^P . A complete schedule fixes four groups of decisions at once: (i) the machine assignment x of every operation; (ii) the operation sequence π on every machine; (iii) the vehicle assignment y of every transport; (iv) a conflict-free path on G for every vehicle movement. We write

$$\mathcal{S} = (x, \pi, y, R),$$

where the reservation table R is the explicit record of group (iv), defined below.

Transport tasks are *induced* by the assignment rather than given as input: a transport arises only when two consecutive operations are not at the same node. A transport splits into two segments—an empty travel to pick up and a loaded travel to deliver—each with its own task label u (the empty and the loaded segment of O_{ji} , for instance, are two separate records). Loading and unloading are instantaneous: the loaded segment departs at the later of the arrival of the empty segment and the readiness of the job, and the delivery instant is at once the instant at which the machine may start and the vehicle may take a new task. Which occupancies may be rewritten later, which keep the original plan, and how rerouting proceeds are all handled at the granularity of a segment, not of a whole operation.

3.1.2 Corridor reservations and conflict freedom. Every corridor $e \in E$ is an exclusive resource with known traversal time τ_e ; the two directions share one occupancy, so at most one vehicle occupies e at any instant, and head-on conflicts and same-direction overtaking are both excluded. Occupancies are half-open intervals: a second vehicle may enter exactly at the right endpoint. Nodes carry ample berths, so waiting happens at nodes and is always permitted. A

(a) operation-precedence graph G_T (vertex = operation) (b) reservation dependency graph G_R (vertex = corridor occupancy)

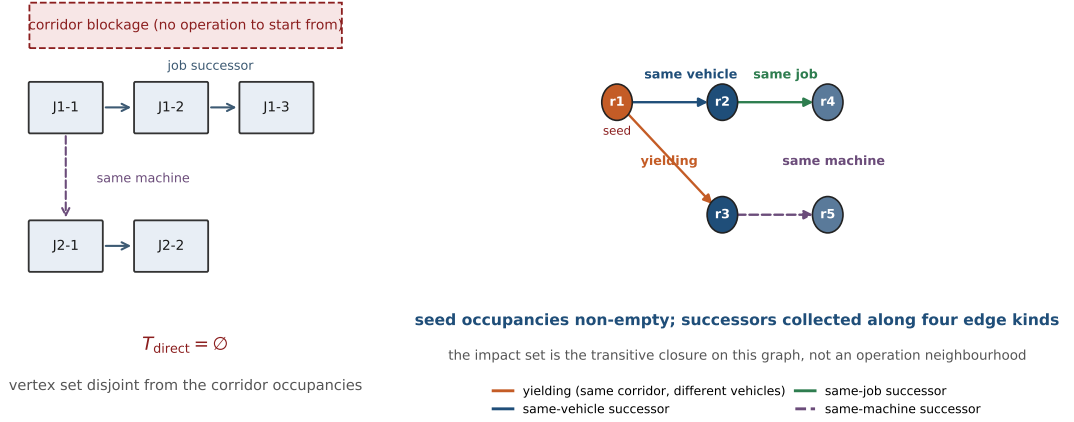


Fig. 2. The vertex sets of the two graphs are disjoint (schematic construction). (a) The operation-precedence graph G_T , whose vertices are operations; a corridor blockage finds no operation to start from. (b) The reservation dependency graph G_R , whose vertices are corridor occupancies; starting from the occupancies overlapping the blockage window, the downstream occupancies are collected along four kinds of dependency edge (yielding, same vehicle, same job, same machine; Definition 4).

reservation is recorded as

$$r = (e, [t^s, t^e], k, u) \in R,$$

that is, vehicle k occupies corridor e during $[t^s, t^e]$ as part of transport segment u . Any two distinct occupancies of the same corridor must satisfy $[t^s, t^e] \cap [t^{s'}, t^{e'}] = \emptyset$. A path is written only into the time windows that R reports as free, so conflict freedom is constructed rather than repaired after the fact.

Time-window routing is the primitive shared by the rerouting below and by every comparison method: given an origin and destination, an earliest departure time and the current R , it returns a path disjoint from the existing occupancies together with the entry time at each corridor. The implementation uses a time-window Dijkstra search [17]; this paper does not modify the primitive and changes only *which* transport segments are handed to the router. Unlike van Os, who reserves nodes, and Lyu, who adds node capacities [10, 20], we reserve edges; Section 7.1 explains the resulting restriction on which instances can be used.

3.1.3 The operation-precedence graph and the reservation dependency graph. The operation-precedence graph $G_T = (T_T, E_T)$ (also called the task graph in the literature, whence the subscript T) has operations as vertices and, as edges, the precedence within a job and the precedence on a machine; it *does not contain* yielding relations among corridor occupancies. The reservation dependency graph $G_R = (R, E_R)$ has the occupancies in R as vertices, and as edges the dependencies induced by yielding, the same vehicle, the same job and the same machine (Definition 4). The vertex sets of the two graphs are disjoint: one is processing, the other is travel. There is therefore a class of disturbance that changes only corridor time windows and no operation parameter—delineating the scope on G_T finds no operation to start from, whereas delineating it on G_R yields a non-empty set of starting occupancies, and the original schedule is already infeasible. Figure 2 contrasts the two graphs, Figure 1 shows the consequence under a corridor blockage, and Table 3 summarizes the notation used in this section and later.

Table 3. Notation used throughout the paper

Symbol	Meaning
$G = (V, E), \tau_e$	corridor network; traversal time of corridor e
$v_0, \text{loc}(m)$	load/unload station; node of robotic arm m
x, π, y	machine assignment, operation sequence, vehicle assignment
$R, r = (e, [t^s, t^e], k, u)$	reservation table; one half-open occupancy (vehicle k , segment label u)
G_T / G_R	operation-precedence graph / reservation dependency graph
$\mathcal{S}^0, \mathcal{S}'$	schedule before the disturbance, schedule after recovery
$\mathcal{D}, t_{\text{now}}$	disturbance event; instant at which it occurs (the decision time)
$T_{\text{direct}}, T_{\text{impact}}$	operations directly disturbed on the operation-precedence graph / scope collected within θ steps
$\text{Seeds}(\mathcal{D}), U$	occupancies used as starting points; set of occupancies allowed to be rewritten
$\text{Cl}(\cdot)$	reservation impact set (STRC)
φ	fraction of future operations disturbed (scale sweep)

3.2 Execution-time assumptions

The execution-time conventions and the physical setting are as follows. A1–A4 are shared by every comparison method, so that the readings can be attributed to how the rescheduling scope is delineated rather than to a difference of models; A5 constrains our method only, the global-rescheduling comparison being allowed a wider set of actions (see the last sentence of that item):

- A1. The initial schedule is feasible.** $\mathcal{S}^0$ passes the conflict-free check before the disturbance, and the reservation table agrees with the operation timetable.
- A2. Finished occupancies are not revisited.** Only unfinished occupancies with $t_{\text{end}} > t_{\text{now}}$ and unfinished operations may be rewritten; the fields of an occupancy with $t_{\text{end}} \leq t_{\text{now}}$ may not.
- A3. A single, fully observed disturbance.** The disturbance is observed in full at t_{now} and does not overlap with another; disturbance streams, prediction error and rolling response are outside the scope of this paper.
- A4. Exclusive corridors, ample buffers, a homogeneous single-load fleet.** Corridors remain exclusive half-open resources, and a blockage is injected into the routing layer as an additional occupancy (or a no-entry window). Finite buffers, charging, acceleration and deceleration, and loading times are not modelled; every vehicle stands empty at v_0 when the plan begins.
- A5. Permitted recovery actions.** A Class-B disturbance keeps the original machine assignment x and sequence π and only reroutes; on failure the set of rewritable occupancies is enlarged. A Class-A breakdown permits two further actions: a vehicle breakdown removes the failed vehicle from the fleet and swaps in another; a robotic-arm breakdown reassigns the unfinished operations to other eligible machines. The sequence π never changes and no search is restarted. The global-rescheduling comparison in the experiments may change x and π (Section 5.3).

3.3 A dichotomy of disturbances

DEFINITION 1 (CLASS-A AND CLASS-B DISTURBANCES). A disturbance is called **Class A** if it directly invalidates the assignment or the executability of some operations (robotic-arm breakdown, marked deviation in processing time, order insertion, vehicle breakdown, and so on); it is called **Class B** if it only makes some corridor time windows unavailable without making any operation unexecutable (corridor blockage, corridor slowdown, and so on).

REMARK 1 (WHY VEHICLE BREAKDOWN IS CLASS A). A vehicle breakdown changes the machine assignment of no operation, and at first sight belongs with corridor blockage. The criterion of Definition 1, however, is executability and not assignment: once a vehicle has broken down, the operations it was carrying cannot start under the original plan, so through the map “vehicle $\rightarrow$ the operations it carries” the operation-precedence graph still has operations to start from. This classification places vehicle breakdown on the side where an operation-level boundary does return a non-empty scope. Those starting operations are non-empty only if the rescheduling method maintains the vehicle–task map; if the method tracks operations and machines only (as is common in implementations of AOR), a vehicle breakdown likewise yields an empty scope. This paper does not rest its argument on that point: Section 6.5 reports the Class-A readings under the stronger convention in which the map is maintained.

DEFINITION 2 (THE DIRECT SET AND THE IMPACT REGION ON THE OPERATION-PRECEDENCE GRAPH). For a disturbance $\mathcal{D}$, let $T_{\text{direct}}(\mathcal{D})$ be the set of operations that cannot run as planned because of a machine breakdown, a vehicle breakdown or an invalidated operation. For a Class-B corridor disturbance we set $T_{\text{direct}} = \emptyset$ by convention. Starting from T_{direct} on G_T and collecting the operations at depth at most θ along job successors and same-machine successors gives $T_{\text{impact}}(\mathcal{D}, \theta)$.

DEFINITION 3 (OCCUPANCIES USED AS STARTING POINTS). The starting set $\text{Seeds}(\mathcal{D}) \subseteq R$ is given per disturbance type and always contains only occupancies with $r.t^e > t_{\text{now}}$.

- (a) **Corridor blockage / slowdown** $\mathcal{D} = (e, [t_a, t_b])$: $\{r : r.e = e, [r.t^s, r.t^e] \cap [t_a, t_b] \neq \emptyset\}$. For several brief blockages one takes the union of the starting occupancies of the individual windows. The two disturbance types share this rule for starting points (the original schedule was written with the unscaled τ , so every occupancy overlapping the window has already been invalidated), and therefore yield the same reservation impact set when the occupancies to be rewritten are delineated—Section 6.5 uses this to explain why their boundary readings are not counted as two independent validations. At the rerouting stage the two are already separated: a slowdown scales progress only on the portion overlapping the window, and is no longer written as a prohibition on the whole segment, nor is an intersecting traversal multiplied by the factor over its whole length.
- (b) **Vehicle breakdown** $\mathcal{D} = (k)$: every occupancy of that vehicle after t_{now} , $\{r : r.k = k\}$.
- (c) **Robotic-arm breakdown** $\mathcal{D} = (m, F)$, with F the operations on that machine not yet finished: for each job take the smallest operation index $i_{\min}(j)$ in F , and let the starting occupancies be all unfinished occupancies of each job from that operation onwards (inclusive). The reason for taking the whole job suffix rather than the operation itself is that the inbound transport may already have finished before t_{now} ; taking the operation alone would make the reservation impact set shorter than the impact region on the operation-precedence graph, and the comparison would be unfair.

PROPOSITION 1 (ON CLASS B THE OPERATION-PRECEDENCE GRAPH RETURNS AN EMPTY SCOPE). Let $\mathcal{D}$ be a corridor blockage. Under the Class-B convention of Definition 2 we have $T_{\text{direct}}(\mathcal{D}) = \emptyset$ and hence $T_{\text{impact}}(\mathcal{D}, \theta) = \emptyset$ for every finite $\theta \geq 0$. If in addition $\text{Seeds}(\mathcal{D}) \neq \emptyset$, then S^0 is infeasible under $\mathcal{D}$.

PROOF. A corridor blockage makes corridor time windows unavailable without making any operation unexecutable, and is therefore Class B by Definition 1. This step uses two premises: the blockage window $[t_a, t_b]$ is finite, and by Assumption A4 buffers are ample so that vehicles can pass outside the window; hence even if the network is cut during the blockage, no operation becomes unexecutable and it is merely delayed. If the blockage were indefinite, or the buffers insufficient to support the waiting, the disturbance would be Class A by Definition 1 and would lie outside the scope of this proposition. Being Class B, it follows from the Class-B convention of Definition 2 that $T_{\text{direct}} = \emptyset$. The starting set of the θ -step collection is empty, so $T_{\text{impact}} = \emptyset$. If some $r \in \text{Seeds}(\mathcal{D})$ exists, its occupancy interval intersects the

no-entry window. Interpreting $\mathcal{D}$ as a compulsory occupancy of that corridor, r and the no-entry window overlap in time on the same exclusive resource, which violates the half-open conflict-free condition (corridor exclusivity in Assumption A4); hence S^0 is infeasible. $\square$

Proposition 1 is a counterexample at the level of the problem. The first half follows directly from the Class-B convention of Definition 2 (the operation-precedence graph contains, by definition, no yielding relations among corridor occupancies and therefore cannot reach a corridor blockage); what is non-trivial is the second half—once the starting occupancies are non-empty, the original schedule is already infeasible, and so “an empty scope on the operation-precedence graph $\Rightarrow$ no rescheduling needed” fails in a setting where corridors are reserved by time windows. The counterexample depends on no repair algorithm; Section 6 reports the sizes of the instances.

3.4 The recovery problem

PROBLEM 1 (BOUNDED REPAIR WITH THE SCOPE DELINEATED ON THE RESERVATION TABLE). *Given a feasible schedule S^0 , a disturbance $\mathcal{D}$ and an instant t_{now} , find S' such that: (i) S' passes the conflict-free check under $\mathcal{D}$; (ii) the corridor, interval and vehicle of every occupancy not brought into the rewritable scope agree with S^0 as far as possible; (iii) the algorithm runtime respects a budget. The secondary objective is the makespan $C_{\max}(S')$.*

Solving it splits into two steps: first delineate the set of occupancies allowed to be rewritten, $U \subseteq R$; then reroute only the transports inside U , keeping the original plan for the occupancies outside it. We give U as $\text{Cl}(\text{Seeds})$; the comparison arm instead induces a subset of reservations from the impact region on the operation-precedence graph.

4 The spatiotemporal reservation closure

This section produces nothing but the set U of occupancies allowed to be rewritten: given the reservation table R and the set of starting occupancies $\text{Seeds}(\mathcal{D})$ (Definition 3), a dependency graph is built on the occupancies and its transitive closure taken, yielding $U = \text{Cl}(\text{Seeds})$; how rerouting proceeds on U belongs to Section 5.

4.1 Reservation dependency edges

DEFINITION 4 (RESERVATION DEPENDENCY GRAPH). *The vertices are the corridor occupancies in R . A directed edge $r \rightarrow r'$ means “the invalidation of r may force r' to be rerouted”, and comprises:*

- (1) **Yield edge:** same corridor, different vehicles, overlapping in time or abutting as half-open intervals ($r.t^e = r'.t^s$), directed from the earlier entrant to the later one; abutment is legal under the exclusivity semantics, but if the former is delayed the latter must yield;
- (2) **Same-vehicle successor:** two occupancies of the same vehicle that are adjacent in time;
- (3) **Same-job successor:** an occupancy of operation i of a job points to an occupancy of operation $i + 1$;
- (4) **Same-machine successor:** an edge between the occupancies induced by two operations adjacent on the time axis of the same machine.

This yields $G_R = (R, E_R)$. The four kinds of edge are illustrated in Figure 2(b). A same-job successor connects the occupancies induced by consecutive operations, and a same-machine successor connects the occupancies induced by two operations adjacent on the time axis of one machine; both have segments as endpoints, matching the granularity of Section 3.1.1. The four kinds of edge answer one question only: after r has been rewritten, may r' have to be rewritten too. They do not attempt

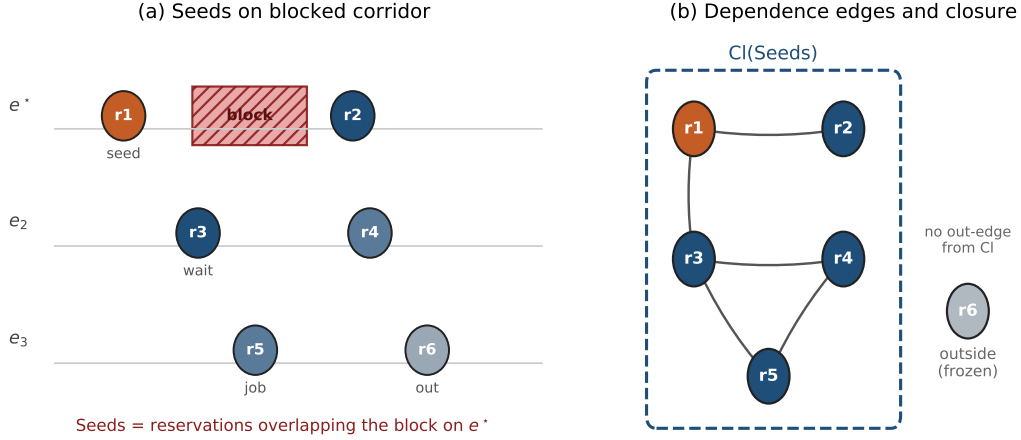


Fig. 3. Construction of the reservation impact set. (a) The occupancies overlapping the corridor blockage window serve as starting points; (b) taking the transitive closure along the four kinds of dependency edge (yielding, plus the three successor kinds: same vehicle, same job, same machine) gives $Cl(Seeds)$, while the occupancies outside the set keep the original plan.

to capture every physical coupling; the edge set is itself a modelling choice, its completeness is tested by the differential probe of Section 6.4, and the corresponding qualifications appear in Section 4.2.

DEFINITION 5 (STRC). For a set of starting occupancies $S \subseteq R$, a horizon H and an instant t_{now} , write the occupancies that have not finished at the decision time and that start before the horizon as

$$R^\circ = \{r \in R : r.t^e > t_{\text{now}}, r.t^s < H\},$$

and let G_R° be the subgraph of G_R induced on R° . The reservation impact set (the spatiotemporal reservation closure) is defined as

$$Cl(S) = \{v \in R^\circ : \exists s \in S \cap R^\circ \text{ such that } s \rightarrow^* v \text{ in } G_R^\circ\},$$

that is, every unfinished occupancy reachable along out-edges from an unfinished starting occupancy (the starting occupancies themselves included). Every experiment in this paper takes $H = C_{\text{max}}(S^0) + 1$, so that $r.t^s < H$ holds for every occupancy of S^0 and the horizon performs no filtering; H is retained only so that the definition remains usable with a rolling horizon. Representing G_R° by adjacency lists (writing its edge set as E_R°), the closure can be computed with a queue in $O(|R^\circ| + |E_R^\circ|)$ time; this bound is not timed separately, since delineating the boundary is only one component of the millisecond-level readings of Section 6.9.

Figure 3 illustrates: (a) the blockage window and the occupancies used as starting points; (b) the dependency edges and the shell of the reservation impact set—an occupancy outside the set is not an out-edge successor of one inside it (Proposition 2).

The comparison arm that delineates the scope on the operation-precedence graph maps T_{impact} into a set of occupancies allowed to be rewritten: every occupancy whose task label belongs to that impact region and whose $t^e > t_{\text{now}}$. By Proposition 1 this set is empty on Class B. When the boundary is drawn on the reservation impact set we take $U = Cl(Seeds)$. In the experimental tables the two are denoted R1 and R2.

4.2 Containment

PROPOSITION 2 (STRUCTURAL CLOSEDNESS). *For any $S \subseteq R$ there is no edge $u \rightarrow v$ in G_R° with $u \in \text{Cl}(S)$ and $v \in R^\circ \setminus \text{Cl}(S)$. Consequently no unfinished occupancy outside the impact set is reachable from $\text{Cl}(S)$ along out-edges.*

PROOF. If such an edge $u \rightarrow v$ existed, then $u \in \text{Cl}(S)$ would give an unfinished starting occupancy s with $s \rightarrow^* u$. Concatenating $u \rightarrow v$ gives $s \rightarrow^* v$, so by Definition 5 we would have $v \in \text{Cl}(S)$, a contradiction. $\square$

Proposition 2 says that “local” corresponds to an object that is closed in the graph-theoretic sense: an occupancy outside the impact set is not the direct successor of any occupancy inside it. This does not claim that G_R has enumerated every coupling possible in the physical world—the edge set is a modelling choice; once the edge set is fixed, closedness is a corollary of the definition rather than a heuristic radius.

REMARK 2 (THE RESERVATION IMPACT SET AND A SMALLEST REWRITABLE SET). *Structural closedness says that the impact set is large enough: no occupancy inside it has a dependency edge leading outside. It does not say that the impact set is not too large. There are three reasons.*

First, a smallest rewritable set depends on the repair procedure used, whereas the reservation impact set does not. To call a set U smallest, one must state “under what repair capability U suffices to restore feasibility”. The smallest U when reassignment and resequencing are permitted is not the same set as the smallest U when only rerouting is. The reservation impact set is uniquely determined by G_R and independent of the repair procedure—so by definition it is not tied to any one heuristic, and by the same token it cannot simultaneously be a smallest set of the kind that depends on the repair procedure.

Second, in the sufficient direction the four kinds of edge remain an over-approximation; in the necessary direction, the one abutment channel we know of has now been closed. A yield edge is created on “same corridor, different vehicles, overlapping or abutting”, that is, whenever the intervals overlap, without judging whether the later vehicle really has no option but to wait for the earlier one—if a bypass exists there, the later vehicle can simply detour and need not be brought into the rewritable set at all; likewise a same-vehicle successor edge may bring in the tail of a chain that the delay never reaches. Both keep the impact set larger than necessary. Abutment was once a missing edge in the opposite direction: half-open intervals that meet end to end are legal, the old rule created no edge, yet if the earlier one is delayed the later one must yield. It is now folded into the yield edges; the differential probe leaks 0 on 50 pairs (0/50; Section 7.1). This closes only the one channel we have observed and is not a proof that the edge set is complete.

Third, as measured, the impact set is not small. Section 6.5 reports that under corridor blockage the reservation impact set covers on average about 90% of the occupancies still unfinished after t_{now} ; in other words, relative to the trivial upper bound of “bringing every occupancy unfinished at the decision time into the rewritable set”, the impact set leaves out less than a tenth. What bounded repair genuinely saves is not rewriting occupancies that have already finished and not restarting the search, not cutting the future down to a small piece. The point is made again in Section 6.8 and among the limitations, and the words “local” and “bounded” should not be read as “only a very small piece is touched”.

“Less than a tenth” is a ratio of the sizes of rewritable sets and is not the same as a difference of rewrite fractions. Section 6.9 implements that trivial upper bound as a separate comparison (RA) and finds that it is precisely this difference of less than a tenth that brings the rewrite fraction down from 0.604 to 0.540 (per cell 31/18/1, $p = 8.8 \times 10^{-7}$), while algorithm runtime and makespan barely change.

What this paper provides, therefore, is a rewritable set that is uniquely determined by the structure, reproducible, and sufficient to restore feasibility, together with closedness; approaching a smallest rewritable set is a different problem and is listed as future work (Section 7.2).

PROPOSITION 3 (A SUFFICIENT CONDITION FOR THE FIELDS OUTSIDE THE SET TO BE UNCHANGED). *Let the set of occupancies allowed to be rewritten be $U = \text{Cl}(S)$, and let the level-1 rerouting run as in Section 5.1: (i) the no-entry window has been written into the routing layer; (ii) for every $r \in R^0 \setminus U$ its original fields $(e, [t^s, t^e], k)$ are pre-committed to the new reservation table and the pre-commit succeeds; (iii) the router is called only for the transports inside U to generate new occupancies; (iv) the final schedule passes the conflict-free check. Then in the recovered reservation table R' the corridor, interval and vehicle of every $r \in R^0 \setminus U$ agree with S^0 . The quantifier here runs over occupancies and not over tasks: one transport may have part of its occupancies inside U and part outside, in which case only the part outside is protected by this proposition.*

PROOF. By (ii), the occupancies outside the set have been written into the new table with their original intervals before the replay begins, and they do not conflict with the no-entry window (otherwise the pre-commit would have failed). Step (iii) submits new occupancies only for the tasks inside U , and neither deletes nor rewrites the already committed intervals outside it. Hence for a task label outside the set, $(e, [t^s, t^e], k)$ in the new table equals the pre-committed value, which is to say it equals the value in S^0 . Passing the check only rules out conflicts between the new occupancies and the existing ones; it does not alter the committed fields. $\square$

Premises (i)–(iv) are execution conditions of the rerouting procedure of Section 5.1 and can be verified against the algorithm one by one; they are not model-level assumptions of the A1–A5 kind and are therefore not folded into the numbered assumption system. The proposition additionally relies on A1 (the initial schedule is feasible, failing which the original fields outside the set might conflict with one another and the pre-commit would necessarily fail), A2 (finished occupancies are not revisited) and A4 (corridor exclusivity).

REMARK 3 (HOW PREMISES (II) AND (III) CAN EACH FAIL). *If the pre-commit fails because “an occupancy outside the set overlaps the no-entry window”, then S failed to cover every occupancy overlapping the blockage window and the starting set is incomplete. If rerouting fails because of a transport–operation handover and thereby triggers the scope escalation of Section 5.2, then the new rewritable set satisfies $U' \supsetneq U$, and Proposition 3 holds only for the occupancies that end up outside the final rewritable set. Escalation puts feasibility ahead of rewriting as little as possible, and is reported separately in the experiments. There is a further failure mode outside the controllable range of the proposition, but easy to produce in an implementation: premise (iii) requires the router to generate new occupancies only for the transports inside U . If the implementation decides whether to reroute at a granularity coarser than an occupancy (a whole transport, say, or a whole operation), then occupancies outside U get rewritten along the way and the conclusion of the proposition ceases to hold—not because the proposition is wrong but because its premise was not met. Remark 4 records an episode in which our own implementation rerouted per whole transport.*

4.3 Structural predictability

The size of the reservation impact set is not a tunable parameter but a quantity uniquely determined by G_R : once the schedule and the network are given, it has no radius left to tune. Section 6.11 scales instances from $8 \times 4 \times 4$ up to $20 \times 10 \times 10$, and the share of the impact set fluctuates between 0.48 and 0.56 without deteriorating with size (this ratio uses the whole table as denominator, and its attainable upper bound is about 0.61; with the rewritable set as

denominator it is 0.78–0.91). “Local” is therefore measurable rather than a radius fixed by hand; but it is not small, see Remark 2.

As to *which* structure corresponds to a larger impact set, the experiments return a negative reading. We first examine a family of measures computed from the network alone, which do not vary with the current schedule: the minimum number of corridors that must be cut to separate the load/unload point from the rest of the shop, that is the load/unload minimum cut, together with the funnel share; below we call this family the static cuts. **The size of the impact set is not determined by the static cuts.** On our own layouts of equal size, the relative sizes under different boundary-blockage protocols either fall within a very narrow band or, where a direction does appear, have heavily overlapping values across random seeds; switching to a batch of networks not designed by us, the load/unload minimum cut and the funnel share *take constant values*, while the share of the impact set moves substantially when three machine configurations (6, 7 and 8 machines) are placed on one and the same network—a constant quantity cannot explain a varying one (Section 6.7). This negative result bears only on the proposition that the static cuts can explain the size of the impact set, and does not amount to “structure is unrelated to the size of the impact set”. The number of public networks available is limited and the latter question remains undecided. The next candidate characterization should be a quantity that varies with the schedule rather than a purely topological one (Section 7.2).

5 Bounded repair on the reservation impact set

Figure 4 gives an overview of this section: the set U of occupancies allowed to be rewritten is first delineated as the reservation impact set; the transports inside it are then rerouted while the original plan is kept outside it; and if feasibility is still not attained, U is enlarged and the attempt repeated. The comparison arm that delineates the scope on the operation-precedence graph replaces only the provenance of U ; the global-rescheduling comparison searches anew within a time limit. All methods share one and the same corridor network, time-window router and conflict-free checker.

5.1 Rewritable, kept as planned, and level-1 rerouting

Given a set U of occupancies allowed to be rewritten, level-1 rerouting keeps the original vehicle assignment and machine assignment and replays in the original sequence. Rerouting is split by corridor leg: a leg a vehicle has already traversed at t_{now} is history and is pre-committed unchanged, and rerouting resumes from the node where the vehicle then stood; a granularity coarser than a leg would rewrite history (Remark 4). Algorithm 1 states this primitive; Algorithm 2 wraps the reservation impact set around it and enlarges U on failure.

The breadth-first pass that delineates the boundary (Definition 5) accounts for only a small part of the runtime: one call of `REPLAYREROUTE` also invokes a time-window shortest path once for each of the $|U|$ transport segments, and the measured runtime is dominated by the latter (Sections 6.4 and 6.9). This level opens no population search. The comparison of Section 6.4 holds Algorithm 1 fixed and replaces only whether U comes from the operation-precedence graph or from the reservation impact set, so as to attribute the difference in feasibility rate to the object the boundary is drawn on.

5.2 Enlarging the rewritable scope after a failure

Enlarging the rewritable scope has precedents of its own. Local Rescheduling [8] widens a time window step by step until feasibility is reached; AOR collects downstream operations along job successors and same-machine successors (Section 2.1). We enlarge U along reservation dependency edges rather than along a time window. These steps serve to

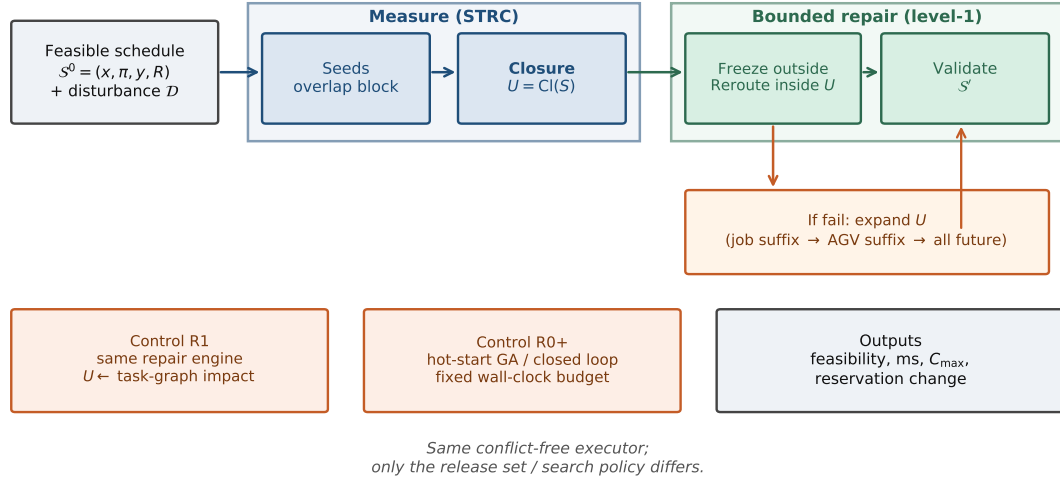


Fig. 4. Bounded repair with the boundary drawn on the reservation impact set. Top row: the impact set is obtained from the starting occupancies and rerouting follows; middle row: the rewritable scope is enlarged after a failure; bottom row: the comparison that draws the boundary on the operation-precedence graph, the warm-started global-rescheduling comparison, and the output measures. The figure shows only the three method-level paths; three further comparisons live in the millisecond band (Section 6.9).

put the impact set to work: when rerouting fails, the occupancies that must be rewritten are still enlarged first, rather than a global search being started at once. Section 6.1 explains why the boundary-ablation experiment must switch this step off.

When keeping the occupancies outside the set is too tight, Algorithm 1 may fail a check such as the transport-operation handover. In that case U is enlarged in the following order and the same rerouting procedure retried: (i) $U \leftarrow U \cup \{\text{the unfinished occupancies of the same-job suffix of each operation currently in } U\}$; (ii) $U \leftarrow U \cup \{\text{the unfinished occupancies of the same-vehicle suffix of the vehicles involved}\}$; (iii) $U \leftarrow \{r \in R : r.t^e > t_{\text{now}}\}$ (bring every occupancy unfinished at the decision time into the rewritable set). After the enlargement it is still a deterministic single-pass rerouting. Experiments show that this step can pull the feasibility rate up to the whole sample while keeping the algorithm runtime at the order of milliseconds to a few tens of milliseconds (Section 6). The step also makes the runtime non-monotone in the disturbance fraction φ , because two effects run in opposite directions: at a small fraction the initial impact set is tight and escalation is needed more often, whereas at a large fraction the initial U is already loose and one round often succeeds, but that single round has more transports to reroute (readings in Section 6.10).

5.3 Comparison with global rescheduling

The global-rescheduling comparison searches anew within a time limit, under the same prohibition constraints and starting from the original schedule (denoted R0+). Internally (x, π) is encoded as a chromosome; one *decoding* means: following the assignment and sequence of that chromosome, call the time-window router of Section 3.1.2 for every transport from the beginning on a reservation table loaded with the no-entry window, obtaining a new R and a new C_{max} . R0+ repeats the decoding and is allowed to change the chromosome; the RD arm of Section 6.9 decodes once and does

Algorithm 1 Level-1 rerouting replay REPLAYREROUTE

Require: schedule $S^0 = (x, \pi, y, R)$, disturbance $\mathcal{D}$, rewritable set U , instant t_{now}
Ensure: schedule S' , or failure

- 1: write $\mathcal{D}$ into the routing layer ▷ blockage: no-entry window; slowdown: τ on the overlapping part
- 2: let R' be an empty table
- 3: **for** every occupancy that may not be rewritten: all those outside the set, and those $r \in U$ with $r.t^e \leq t_{\text{now}}$ **do**
- 4: pre-commit the original fields $(e, [t^s, t^e], k)$ to R' ▷ the second kind is the history of A2; omitting it amounts to rewriting history
- 5: **if** it conflicts with the blockage window **then**
- 6: **return** failure ▷ the starting occupancies did not cover those overlapping the window
- 7: **end if**
- 8: **end for**
- 9: **for** every $r \in U$, in the original sequence **do**
- 10: resume from the node of that vehicle at t_{now} ▷ the traversed legs were already committed above and are not submitted twice
- 11: call the time-window router on the current R' and submit the new occupancies ▷ finished occupancies are not rewritten
- 12: **end for**
- 13: advance the idle times of the machines and the ready times of the jobs according to π
- 14: check independently: conflict freedom, no residue in a no-entry window, the slowdown absorbed on the overlapping part, fields outside the set unchanged
- 15: **if** the check passes **then**
- 16: **return** $S' = (x, \pi, y, R')$
- 17: **else**
- 18: **return** failure
- 19: **end if**

not change the chromosome. The decoding of R0+ does not rewrite occupancies that have already finished: those are pre-committed, and only the operations not yet started are scheduled from t_{now} onwards (Section 6.8). The comparison is warm-started rather than searching from scratch, so as not to discard the assignment and path information already present in the original schedule. R0+ may reassign and resequence and is therefore more likely to shorten C_{max} , but its algorithm runtime is set by the budget.

Bounded repair is not better than R0+ on makespan. Section 6.8 shows that R0+ is already ahead at the smallest budget level, and since R2 is a single fixed point while R0+ is non-increasing in the budget, adding budget produces no crossover either. The strength of bounded repair is its algorithm runtime: when the response must be at the order of milliseconds, it can restore feasibility; when a stoppage of seconds is permitted, R0+ should be used to shorten C_{max} . That the corridor, interval and vehicle of the occupancies outside the set keep the original plan is guaranteed only under the conditions of Proposition 3, and is observed partially in the experiments.

Comparing against R0+ alone leaves one question open: within the millisecond band, is a boundary drawn on the impact set needed at all? Implementing “do not restrict the scope by the impact set” directly answers it, and we do so in three ways: a global right shift (RS) decides nothing about which occupancies are affected and merely pushes the unfinished occupancies as a whole past the end of the blockage; re-decoding of the original chromosome (RD) does not change the chromosome and runs a single pass of fixed-prefix decoding; **RA** brings every occupancy unfinished after t_{now} into the rewritable set and then runs *exactly the same* rerouting procedure as Section 5.1, so that it differs from our method in the single item “which occupancies may be rewritten”—it is the only comparison that isolates the

Algorithm 2 Bounded repair based on the reservation impact set (level-1 rerouting + scope escalation)

Require: schedule S^0 , disturbance $\mathcal{D}$, instant t_{now}

Ensure: schedule S' , or failure

1: $S \leftarrow \text{Seeds}(\mathcal{D}); U \leftarrow \text{Cl}(S)$

2: **for** round = 0, 1, 2, 3 **do**

3: $S' \leftarrow \text{REPLAYREROUTE}(S^0, \mathcal{D}, U, t_{\text{now}})$

4: **if** S' is feasible **then**

5: **return** S'

6: **end if**

7: update U by the escalation rule

▷ job suffix / same vehicle / all unfinished

8: **end for**

9: **return** failure

reservation impact set on its own. All three are at the order of milliseconds and constitute comparisons for bounded repair *within the same runtime band*; their definitions and readings appear in Section 6.9.

Algorithm 2 puts this section and the previous one together, matching Figure 4.

6 Experiments

6.1 Protocol

Network, router and instances. The lower-level network, the time-window router and the independent checker are fixed. The expanded matrix uses **five instances**: the small example `example_3x3x2`, the congested example `congested_8x4x4`, and three `S8x4x4` layouts `high`, `funnel` and `mid`. The last three have the same size and the same flexibility and differ only in corridor topology, so differences in the size of the reservation impact set among them can be attributed to topology rather than to problem size. The five come from two sources: the first two are hand-written layouts (the small example for step-by-step checking, the congested one being a dumbbell topology), while the last three are produced by our instance generator, whose parameters—jobs $\times$ machines $\times$ AGVs, congestion level, H , F —are encoded in the instance file name together with the **generation seed**; all three were generated with seed 42. **The generation seed and the run seeds below are two different things**: the former fixes the instance itself, the latter fix the disturbance injection and the search process; that both happen to take the value 42 is a coincidence, and nothing is shared. **Ten random seeds are used**, namely 42, 7, 2024, 99, 123, 13, 1, 777, 31415 and 8, giving $5 \times 10 = 50$ instance–seed pairs. Every per-cell comparison, win/loss count and paired test in this paper is aligned on the key (instance, seed): under one key all methods read the same initial solution, the same blockage and the same decision time. The initial schedule is obtained by replaying the time-window router of Section 3.1.2 in sequence order, and it passes the independent conflict-free check; under one instance–seed pair all comparison methods share that initial solution. This paper does not depend on how S^0 was found, only on its being feasible before the disturbance.

Implementation and environment. Every experiment is implemented in pure standard-library Python with no third-party numerical package; the lower-level network, the time-window router and the independent checker reuse the same implementation as the static integrated work. Runs are single-machine and single-threaded with no parallelism: Intel Xeon W-1270 (8 cores, 16 threads), 64 GB of memory, Windows 10, interpreter CPython 3.7.6 (64-bit). Since no third-party numerical package is involved, reproduction does not require pinning dependency versions; 3.7 is the interpreter we measured, and higher versions were not re-run one by one. All timing readings were obtained under the

single-machine, single-threaded conditions above, and a different machine would shift them all, so every conclusion about runtime in this paper is stated as a *ratio* or an *order of magnitude* and never argued from absolute milliseconds.

Code and data availability. The material needed for reproduction comes in three parts: the upper-level bounded repair together with the experiment driver, the lower-level network and time-window router, and the instances, disturbance inputs and all per-cell readings (CSV). All three are submitted together with the scripts that regenerate every table and figure, and are provided during review as an **anonymous repository** (see the link attached in the submission system; the repository contains no author-identifying information); upon acceptance it becomes a public repository registered with a DOI on a permanent archiving service, and this section will then be completed with that DOI. Every table and figure can be traced in the repository to the script and the CSV that produced it, and the instances listed in Table 13 are distributed with the repository as well.

Second batch: externalizing the provenance of the layouts. The layouts of the five instances above were all designed by us, and three of them, of equal size, further belong to one dumbbell family (they differ only in the exit of the load/unload point and in the number of parallel lanes in the middle, that is, in capacity and not in geometry). So that the layouts are no longer of our choosing, a second batch of 5 instances was run: the grid size of the network, the placement of the load/unload station and of the machines, and the removed edges were transcribed item by item from the layout figures of Appendix A of Lyu et al. [10], and the transcription was reconciled field by field, by script, against the machine-readable encoding supplied with the van Os toolset. The number of machines is therefore fixed by the layout rather than chosen by us, and ranges from 4 to 8. The same ten random seeds are used, giving 50 instance–seed pairs in total. The two batches are *matched parameter for parameter*: the number of jobs, the number of AGVs, the number of operations per job, H , F and the calibration target of the transport-to-processing time ratio are all identical, so the difference between the batches can be attributed cleanly to the provenance of the layouts.

That appendix gives six layouts in all (3 to 8 machines). The 3-machine one is not used: with $F = 0.6$ fixed, $|M| = 3$ gives an average of $1.8 < 2$ eligible machines, which is incompatible with “at least two eligible machines per operation”, and relaxing F for a single layout would introduce one more variable.

The remaining five **live on only 2 distinct networks**, which must be stated first: the 4-machine and 5-machine layouts share one 4×4 grid (with the same single edge removed), and the 6-, 7- and 8-machine layouts share one 5×5 grid (with the same two edges removed); the five differ in the number of machines and in *where the machines sit*, not in network geometry. This is how the original release itself is organized and not a choice of ours. Hence when Section 6.7 reads this batch, **the number of independent samples along the dimension of network geometry is 2 and not 5**, whereas along the dimension of machine placement there are 5; the statements below follow this division strictly.

Three conventions must be declared up front. **First, the per-leg travel times are not original data.** Lyu published per-leg durations for a single example instance only, and those values are not uniform; the per-leg durations of the Appendix-A layouts have never been published. They are therefore filled in here as “all edges of equal weight”—structurally consistent with the assumption of a constant single-step duration in the van Os model, differing only in the scale of the time unit. **Consequently the results of this batch must not be compared in any way with the reference values of Lyu or of van Os**; what it provides is a set of topologies not designed by us, not a benchmark to be matched. **Second, the load/unload points were merged.** The original layouts have separate loading and unloading stations, whereas we have a single load/unload point, so the loading station at which the vehicles start plays that role and the unloading station degenerates into an ordinary grid node. **Third, the job data were not borrowed along with the layouts.** In the public families we have measured, the mean transport duration is only a few per cent of the mean processing duration, so transport barely produces contention there; importing the job data as well would

degenerate corridor contention, and with it the whole set of conclusions about it, to the point of being unobservable. Only the network is externalized. A further family of layouts shipped with the same toolset (its directory is labelled Liu 2023) is not included in this batch: the data statement of those layouts permits diagonal movement, whereas our corridors are 4-adjacent; and accepting diagonal movement would require changes to the lower-level router rather than to the instances.

Disturbances. Unless noted otherwise, $t_{\text{now}} = 0.35 C_{\text{max}}(S^0)$. E1–E3 and E5 impose a single-window blockage on a busy corridor; E6 adds three further disturbance types at the same t_{now} ; the scale sweep sorts the future operations by their earliest future reservation and takes the reservation windows of the leading fraction φ as micro-blockages (without merging them).

Comparison methods. R1: the boundary drawn on the impact region of the operation-precedence graph + level-1 rerouting, with $\theta = 2$ throughout (the value affects only the size of $|U_{R1}|$ under Class-A disturbances; by Proposition 1 every finite θ yields the empty set on Class B); its delineation rule is taken from the operation-level local rescheduling described in Section 2.1 and reimplemented by us. Item by item, the delineation rule “take the affected operation set to be the directly disturbed operations and extend it backwards along the precedence within a job” corresponds to AOR [1, 16]; whereas *truncating the extension at θ hops* and *mapping the operation set into the corridor reservations that must be released* are both fixed by us, since the original settings have no reservation table and this mapping has no counterpart there; the rerouting and replay components are shared with R2, so that the two differ only in the object the boundary is drawn on. R1 is therefore the comparison obtained by “moving that delineation rule into the reservation setting of this paper” and is **not a reproduction of the cited work**; its readings in Section 6.4 say only that the delineation rule returns an empty scope in this setting, and constitute no evaluation of the original method in its own setting. R2/STRC: the boundary drawn on the reservation impact set + level-1 rerouting (the scale sweep switches scope escalation on; E3 and E5 switch it off). R0+: global rescheduling that starts from the original schedule and searches anew within a time limit—a genetic algorithm with population 40, the chromosome of the original schedule placed in the initial population, and decoding using the same fixed-prefix decoding as RD (occupancies before t_{now} may not be rewritten); *no generation cap and no early-stopping threshold* are set, the only termination condition being the wall-clock budget, whose values are given in Section 6.8. RS: the global right shift, which draws no boundary on the impact set and does not reroute (Section 6.9). RD: re-decoding of the original chromosome, that is the cheaper global recomputation in this implementation (same section). RA: no restriction of the scope by the impact set, but rerouting retained—every occupancy unfinished after t_{now} is brought into the rewritable set and everything else is identical to R2, so as to isolate the reservation impact set on its own (same section). These five comparisons span the plane “is a boundary drawn $\times$ how much is recomputed”: R1 and R2 differ only in the *object* the boundary is drawn on, RA and R2 differ only in *whether* a boundary is drawn at all (the rerouting procedure being identical), RS neither draws a boundary nor reroutes but pushes everything back, and RD and R0+ recompute globally without drawing a boundary on the impact set, differing only in how many generations they search.

Budget conventions. The methods are not compared at a common budget level, and this must be said first: R1, R2, RS, RD and RA are all one-shot deterministic repairs that stop when done, their runtime being fixed by instance size and impact-set size rather than by a budget setting; only R0+ terminates on a wall-clock budget. There is therefore no alignment in wall clock, generation count or evaluation count between R0+ and the other methods, and every reading involving R0+ is to be understood as “the reading at the given budget”; only the four-method comparison of Section 6.9 is a comparison within one budget level (all of them stop when done). Wall-clock runtimes are reported per batch and are not read across batches.

One convention that must be stated clearly. Scope escalation *must be switched off* in E3. Left on, R1 would succeed incidentally by “escalating all the way until every unfinished occupancy is in the rewritable set”, and what would be measured is no longer the difference between the objects the boundary is drawn on but the fallback capability of the escalation rule. Escalation itself works (it pulls the feasibility rate to the full sample in the scale sweep), but it is orthogonal to the variable under test, and is therefore switched off throughout the boundary ablation. That endpoint is not thereby avoided: it is exactly the RA arm of Section 6.9, where its cost is measured cell by cell as an independent comparison.

What each of the two batches is responsible for. The two batches are not two samples of the same body of evidence; they answer different questions, and each section of this chapter is supported by one of them only. The main batch answers *whether the phenomenon exists and whether the mechanism is clean*: under a Class-B disturbance the boundary drawn on the operation-precedence graph returns an empty scope while the original schedule is infeasible (Section 6.2); the reservation impact set contains that scope and the fields outside the set are invariant (Section 6.3); and under one and the same rerouting procedure only the object the boundary is drawn on is replaced (Section 6.4). All three require complete control over the instance parameters, especially the transport-to-processing time ratio—which decides whether corridor contention exists at all—and the public families cannot provide such control (Section 7.1, limitation 4). The second batch answers a different question: *whether these phenomena hold only on the layouts we chose ourselves*. It is therefore used only for reproduction and produces no new performance readings (Section 6.12).

Two points deserve emphasis. First, **the second batch is used to test whether the phenomena hold only on the layouts we chose ourselves**, and to narrow the conclusion on structural predictability (Section 6.7). Second, **only the single item E4 must be read across both batches**: the three equal-size layouts of the main batch belong to one family and their geometric differences are small, so on them the two explanations “the measure is too coarse” and “the instance set does not differ enough geometrically” are confounded; and the external layouts, without a parameter-matched self-designed control, give no way to judge whether the movement in their impact-set share is large enough. The conclusions of the other sections stand on the main batch alone, and the second batch only strengthens their external validity.

Statistical conventions. There is exactly one inferential test in this paper: the difference in rewrite fraction between R2 and RA in Section 6.9, using a two-sided Wilcoxon signed-rank test, paired on the (instance, seed) key described above. Because this is the only test performed, there is no multiple comparison and no correction was applied; the per-cell win/loss counts, means and medians elsewhere are all *descriptive* statistics and no significance is asserted for them. The accompanying effect size is reported as a Hodges–Lehmann estimate of 0.037, with a 95% bootstrap confidence interval [0.018, 0.074] (percentile method, 10^4 resamples, fixed random seed); the mean of the paired differences is 0.064, interval [0.039, 0.093]. Dispersion is reported as an interquartile range: for the rewrite fraction it is 0.073 under R2 and 0.062 under RA. Because 18 of the 50 cells happen to tie between the two methods, the lower end of the interval for the *median* paired difference touches 0—this is recorded as it stands, and it shows that ties make up a third of the win/loss count. The tables of the other batches give means, medians and counts only, without intervals.

6.2 E1: the operation-precedence graph returns an empty scope

This section asks one thing only: what scope each of the two ways of drawing the boundary returns under a Class-B disturbance, with no comparison of repair quality. Table 4 and Figure 5 (left) summarize the expanded results of Proposition 1 under the corridor-blockage protocol of Section 6.1: on 50/50 of the samples $T_{\text{impact}} = 0$, the starting occupancies and the reservation impact set are non-empty, and the original schedule is infeasible after the blockage.

Table 4. E1, the operation-precedence graph returns an empty scope under corridor blockage (10 random seeds per instance, 50 pairs in total). $|R|$ is the number of occupancies in the whole table and $CI/|R|$ the share of the reservation impact set in it. Means are taken over the seeds. Dispersion is reported as an interquartile range (IQR): the per-instance IQR of $CI/|R|$ lies between 0.024 and 0.049, the median over the 50 pooled pairs is 0.571, and the spread is 0.500–0.655; the between-instance and between-seed variation of this share are thus of the same order, and the mean alone should not be read. “C1 pass” counts the cells on which the three conditions of Proposition 1 hold simultaneously ($T_{\text{impact}} = 0$, non-empty starting occupancies, and the original schedule infeasible after the blockage). The last three rows have the same size and the same flexibility and differ only in corridor topology.

Instance	Seeds	C1 pass	mean $ R $	mean $ Seeds $	mean $ CI $	$CI/ R $
example_3x3x2	10	10/10	28.6	5.6	16.9	0.59
congested_8x4x4	10	10/10	85.0	15.0	48.4	0.57
S8x4x4 (high)	10	10/10	150.5	15.5	88.3	0.59
S8x4x4 (funnel)	10	10/10	149.1	15.5	85.4	0.57
S8x4x4 (mid)	10	10/10	147.6	15.0	82.8	0.56
Total	50	50/50	—	—	—	—

As the paragraph following Proposition 1 warns, the weight in reading this table should fall on the *second* clause. Table 4 folds three conditions into a single “C1 pass” column and counts them together, but the three are not of the same nature and must be read apart; the per-cell readings are in the columns n_T_impact and $feasible_after_block$ of the data file `e1_miss.csv`. Among them, $T_{\text{impact}} = 0$ is a corollary of the Class-B convention of Definition 2: it makes visible that “the operation-precedence graph cannot, by definition, reach a corridor blockage”, but it is not itself a finding. What is non-trivial is that the latter column is uniformly false—the schedule *is* already infeasible. Only the two together constitute the counterexample.

The absolute value of $|CI|$ grows with the instance (16.9, 48.4, 82.8–88.3), a direct consequence of there being more occupancies in total; $CI/|R|$, by contrast, falls between 0.56 and 0.59 on all five instances and *fails* to distinguish the three S8x4x4 layouts. This agrees with the conclusion of Section 6.7: the experiments provide no structural measure that reliably predicts the size of the impact set. As far as the contributions are concerned, this section establishes one thing only: under a Class-B disturbance the boundary drawn on the operation-precedence graph returns an empty scope while the original schedule is indeed infeasible; the particular values of $CI/|R|$ carry no claim.

6.3 E2: containment

Structural containment (E2a) holds on 50/50 of the samples (leakage = 0), in agreement with Proposition 2. The level-1 repair (without escalation) is feasible on 50/50 of the samples. That the corridor, interval and vehicle of every occupancy outside the set are unchanged entry by entry (E2b, the observational form of Proposition 3) likewise holds on 50/50 of the samples (Table 5).

E2a and E2b must be kept apart, and *a full pass does not promote Proposition 3 to a necessary and sufficient condition*. E2a is a property *of the graph*, guaranteed by Proposition 2, and a full pass is what ought to happen; were it not a full pass, the implementation would disagree with the definition. E2b is a property *after the repair*, and what it tests is whether the four premises of the proposition hold under our protocol—not that those premises may be dispensed with. In particular, premise (ii) requires that every occupancy outside the set be pre-committed successfully; if one of them overlaps the no-entry window the pre-commit fails, which indicates that the set of starting occupancies is incomplete (Remark 3), and the proposition does not apply. Our experiments fall on the side where the premises hold, which is not to say that any setting will fall on that side. To close this out in terms of the contributions: E1, E2 and the boundary

Table 5. E2, containment (10 random seeds per instance; the level-1 feasibility column uses no escalation). E2a is structural closedness (leakage = 0), and E2b is that the corridor, interval and vehicle of the occupancies outside the set are unchanged *entry by entry*.

Instance	E2a closed	Level-1 feasible	E2b outside unchanged
example_3x3x2	10/10	10/10	10/10
congested_8x4x4	10/10	10/10	10/10
S8x4x4 (high)	10/10	10/10	10/10
S8x4x4 (funnel)	10/10	10/10	10/10
S8x4x4 (mid)	10/10	10/10	10/10
Total	50/50	50/50	50/50

ablation of Section 6.4 support the layer of *the object standing up*—the reservation impact set is uniquely determined by the edge set, closed outwards, and sufficient to restore feasibility under one and the same rerouting procedure; how much benefit drawing the boundary on this object buys over not drawing it is *another layer*, supported only by the RA comparison of Section 6.9.

REMARK 4 (AN EPISODE IN WHICH AN IMPLEMENTATION DEFECT WAS READ AS A DEFECT OF THE EDGE SET). *This column stood at only 24/50 in an earlier batch. The reading at the time was that “the dependency edge set fails to cover certain weak couplings”, and refining the edge set was accordingly listed as the first item of future work. On review that reading was wrong. What may be rewritten is a set of occupancies, whereas at replay time the decision to reroute was taken per operation—as soon as either the empty or the loaded segment of an operation fell inside the impact set, the whole transport was replanned. So when only the loaded segment entered the impact set, the empty segment of the same operation was rewritten as well, even though its task label was not in the rewritable set, and this was recorded as drift outside the set. The 84 drifting occupancies on the 26 failing cells were without exception empty segments of this kind, and 71 of them (85%) had already been executed before t_{now} —rewriting such a segment also violates “do not rewrite occupancies that have already finished”. After the decision was moved to a per-segment basis (an empty segment already executed or not brought into the rewritable set keeps the original plan, and only the segments genuinely inside the rewritable set are rerouted), the drift fell to zero. In other words, what had been measured was not the coverage of physical couplings by the edge set but the fact that premise (iii) of Proposition 3, “call the router only for the transports inside U ”, had not really been met in the implementation. We record this because it exemplifies an easy misjudgement: reading a granularity mismatch between implementation and definition as an insufficiency of the model with respect to the world.*

One more layer of the same mismatch remains, and E2b cannot detect it. Even after the decision is taken per segment, a transport segment that straddles t_{now} is still replanned in its entirety: as soon as the end time of one of its corridor legs falls after t_{now} , the task label of the whole segment enters the rewritable set, and so the legs it had already traversed before t_{now} are rescheduled too—which amounts to sending the vehicle back to the start of the segment to travel it again, in violation of “do not rewrite occupancies that have already finished”. E2b cannot detect this, because the task labels of those legs are inside the impact set and hence outside the comparison range of the fields outside the set. It took an independent audit of “finished occupancies may not be rewritten” (comparing entry by entry the occupancies with $t_{\text{end}} \leq t_{\text{now}}$) to measure it: 36 of the 50 cells contained rewrites of this kind, 86 occupancies in all. After the granularity was refined by one more level—splitting by corridor leg, keeping the traversed legs unchanged, and resuming the rerouting from the node at which the vehicle actually stood at t_{now} —that audit fell to zero on all 50 cells, with every pass rate unchanged, C_{max} improving on 6 cells and worsening on none. The readings reported in this paper are the corrected ones. The historical readings in this remark (24/50; 84 occupancies on 26 cells; 86 occupancies on 36 cells) come from the batches before the

Table 6. E3, boundary ablation (no escalation; 5 instances $\times$ 10 random seeds). miss_B means that on that cell the rewritable set of R1 is empty while that of R2 is not. The last column is the mean over the 10 seeds within each instance, a different convention from the per-cell median in the body.

Instance	miss_B	R1 feasible	R2 feasible	mean $ U_{R2} $	R2 runtime/ms
example_3x3x2	10/10	0/10	10/10	16.9	0.66
congested_8x4x4	10/10	0/10	10/10	48.4	2.84
S8x4x4 (high)	10/10	0/10	10/10	88.3	3.40
S8x4x4 (funnel)	10/10	0/10	10/10	85.4	5.09
S8x4x4 (mid)	10/10	0/10	10/10	82.8	3.96
Total	50/50	0/50	50/50	—	—

correction and from one independent audit of “finished occupancies may not be rewritten”; they appear in none of the tables of this paper and support no claim.

6.4 E3: boundary ablation

Table 6 and Figure 5 (right) hold the level-1 rerouting fixed and *switch off* scope escalation. On Class B, the rewritable set of R1 is empty and infeasible on 50/50 of the samples, while R2 is feasible throughout.

This is the strongest comparison in the paper, but its force comes from *differing in one item only* and not from the size of the numbers: the same rerouting procedure, the same initial solution, the same t_{now} , the same random seeds and the same blockage window, with **the sole difference being which occupancies may be rewritten**. The contrast 0/50 against 50/50 therefore cannot be attributed to the quality of a rerouting heuristic—on the R1 side no rerouting is triggered at all, its rewritable set being empty, and level-1 rerouting is the identity on the empty set, so the original schedule stays untouched on the blocked corridor. Put differently, R1 fails not by “repairing badly” but by “judging that no repair is needed”. The per-cell runtime of level-1 rerouting has median 3.2 ms and maximum 12.8 ms; the last column of Table 6 is the mean over the 10 seeds within each instance, a different convention, and the two must not be read against each other. As far as the contributions are concerned, this section pins the cause of the gap on the object the boundary is drawn on: replacing only that object under one and the same rerouting procedure moves the rate at which feasibility is restored from 0/50 to 50/50; this still belongs to the layer of *the object standing up* and says nothing about the net benefit of the boundary.

Whether the contrast above depends on the calibration of the edge weights is tested separately. Our corridor weights are uniform, so E1 and E3 were re-run on the same 50 pairs under three non-uniform weight settings (lognormal, `lu_far`, `split`; data in `weight_sweep.csv`): the readings for C1 pass and for the empty operation-precedence scope are unchanged at every setting, R1 remains infeasible on every cell, and the number of cells on which level-1 R2 is feasible is 50/50, 50/50 and 49/50 respectively, the single infeasible cell occurring in the `split` setting (S8x4x4_high, seed 1). The contrast therefore does not depend on the uniform calibration; but a sweep over self-assigned edge weights still cannot replace an instance family that publishes both the network and the per-leg durations (Section 7.1, limitation 4).

6.5 E6: crossing disturbance types with boundary definitions

E1–E3 used corridor blockage as the only disturbance. This subsection opens up the dimension of disturbance type, adding corridor slowdown, vehicle breakdown and robotic-arm breakdown on the same 50 pairs, and gives Table 7.

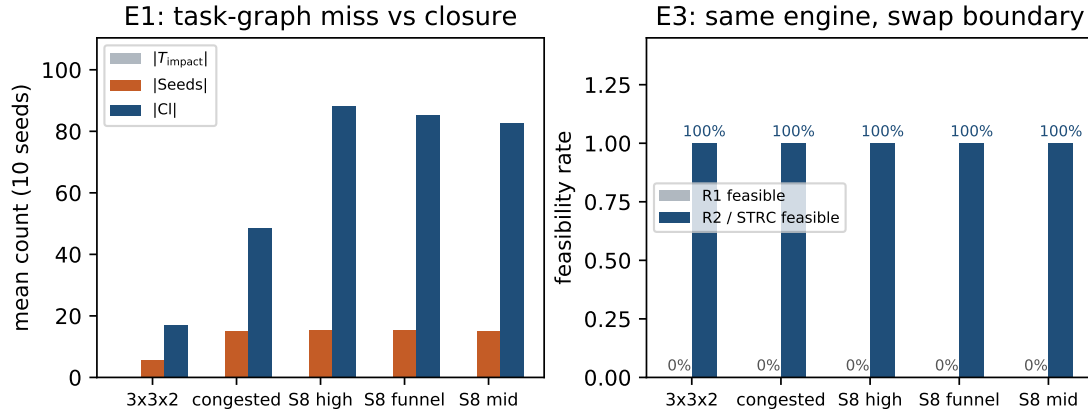


Fig. 5. Visualization of the expanded E1/E3 results (5 instances $\times$ 10 random seeds, means). Left: the impact region on the operation-precedence graph is empty while the starting occupancies and the reservation impact set are not; bar heights are the means over the 10 cells of each instance, and the dispersion as an interquartile range is given in Table 4 (the per-instance IQR of $|CI|$ lies between 2 and 8; the impact-region series is identically zero and has no dispersion). Right: with the same rerouting and only the object the boundary is drawn on replaced, the feasibility rate of R1 is 0 and that of R2 is 100%; this panel is a per-cell count (50 cells each), not a mean, and therefore carries no dispersion.

Table 7. E6, disturbance type $\times$ the object the boundary is drawn on (50 pairs per type, 200 measurements in all; two of the types are Class B and yield the same starting set by construction, so the number of *independent* boundary measurements is 150, see point three in the body). $|R^\circ|$ is the number of occupancies unfinished after t_{now} . “R1 empty” counts the cells on which the boundary drawn on the operation-precedence graph declares that the disturbance needs no rescheduling. The last column uses no escalation: level-1 rerouting for Class B, vehicle swap or reassignment for Class A.

Disturbance	Class	R1 empty	mean $ U_{R1} $	mean $ CI $	$CI/ R^\circ $	R2 feasible
Corridor blockage	B	50/50	0.0	64.4	0.92	50/50
Corridor slowdown	B	50/50	0.0	64.4	0.92	50/50
Vehicle breakdown	A	0/50	31.1	60.6	0.85	50/50
Robotic-arm breakdown	A	0/50	42.8	65.9	0.94	50/50

What is measured, first. On the boundary side, all four types are reported: how large the impact region on the operation-precedence graph is, how large the reservation impact set is, and whether the latter contains the former. On the repair side, all four types are opened up. Class B still reroutes only: a blockage writes a no-entry window into the reservation table, while a slowdown reduces progress inside the window by a factor $\tau_{\text{mult}} = 2$ and keeps the original τ outside it, the corridor remaining passable. Class A additionally swaps vehicles or reassigns machines, without changing the sequence; whether the rewritable scope is enlarged does not change the action set for them.

Four readings. First, **the A/B dichotomy holds without exception on the 150 *independent* boundary measurements** (the two Class-B disturbances share one starting set by construction, see point three below, so they are not counted as 200): both Class-B types return an empty scope on the operation-precedence graph, and both Class-A types return a non-empty one. Second, **$U_{R1} \subseteq CI$ on all 100 Class-A measurements**—so moving to a boundary drawn on the reservation impact set *loses* none of the occupancies that the operation-precedence graph already included. The containment is strict on 87/100 of the cells, while on the remaining 13 the two are exactly equal (there the impact set

does not leave, along dependency edges, the set induced by the impact region on the operation-precedence graph), which is *not* a containment failure. Among these, the containment on the 50 robotic-arm-breakdown cells holds *by construction*: the starting rule of Definition 3(c) takes the whole suffix along the job chain, whereas U_{R1} truncates at $\theta = 2$ (Section 6.1), so what that row verifies is agreement between implementation and definition rather than an independent finding; the containment on the 50 vehicle-breakdown cells is not given by construction and is a measured result. The price is that the rewritable set becomes larger on average: $31.1 \rightarrow 60.6$ for vehicle breakdown and $42.8 \rightarrow 65.9$ for robotic-arm breakdown. Together with the first reading, the reservation impact set is a uniform strengthening in *coverage* over the operation-precedence boundary on these four disturbance types—no occupancy already included is lost; it does cost something in the size of the set, so the two are not a trade-off in coverage. Third, **the two boundary rows for slowdown and blockage are identical cell by cell**, with the same impact-set size on all 50 pairs. This is not two independent validations of the boundary: for both, the starting rule is “the unfinished occupancies overlapping the invalidated window”, so by construction they yield the same starting set and hence the same reservation impact set. What that row does confirm is that the boundary drawn on the operation-precedence graph is likewise empty for a slowdown. Fourth, **the repair side is full on all four types**, with the per-type readings in the last column of Table 7, slowdown and blockage sharing one convention. The slowdown once multiplied any traversal intersecting the window by 2 over its whole length, and 13 cells died on “the occupancy is too short”—when there was no free slot for 2τ inside the window, the entry time was pulled back into the window while the occupancy was still committed with the original τ ; 12 of them could not pass even with every unfinished occupancy brought into the rewritable set, so it was not keeping the plan outside the set that stranded the slowed vehicle. After the change to scaling only the overlapping progress, level 1 passes fully and escalation is no longer triggered. The corridor remains passable, so the physical difference from “seal the whole segment and detour” is still there; it is merely no longer amplified by a factor applied to the whole segment. Vehicle breakdown is handled by a vehicle swap, robotic-arm breakdown by reassignment. Vehicle swap and reassignment are the actions that implement Class-A recovery, and serve to complete the repair side of the coverage matrix.

This table also gives the measured size of the impact set (Remark 2). Under corridor blockage the reservation impact set covers about 92% of the occupancies unfinished after t_{now} ; relative to the trivial upper bound of “bring every unfinished occupancy into the rewritable set” it leaves out less than a tenth. What bounded repair genuinely saves is not rewriting occupancies that have already finished, and not restarting the search—not cutting the future down to a small piece. The $Cl/|R|$ of Section 6.2, computed over the whole table and falling in 0.56–0.59, does not contradict this: the whole table includes the half already executed before t_{now} , which was never modifiable in the first place. Both conventions are given. As far as the contributions are concerned, this section extends only *coverage*—carrying the A/B dichotomy and the containment from one disturbance type to four; it strengthens no claim and introduces no new object for the boundary to be drawn on.

6.6 A case Gantt chart

E1–E3 give counts and feasibility rates across random seeds. To place “beyond the reach of the operation-precedence graph, repairable via the reservation impact set” on the time axis, Figure 6 takes one corridor blockage on example_3x3x2 with seed 42, under the same protocol as E2/E3: the decision time is $0.35 C_{\text{max}}$, a single-window blockage on a busy corridor, level-1 rerouting and no escalation.

On this instance $T_{\text{impact}} = \emptyset$, while $|\text{Seeds}| = 6$ and $|Cl| = 18$. Figure 6(a) is the machine Gantt chart of the original schedule before the disturbance: the transport occupancies of the blue operations fall inside the impact set (and will be

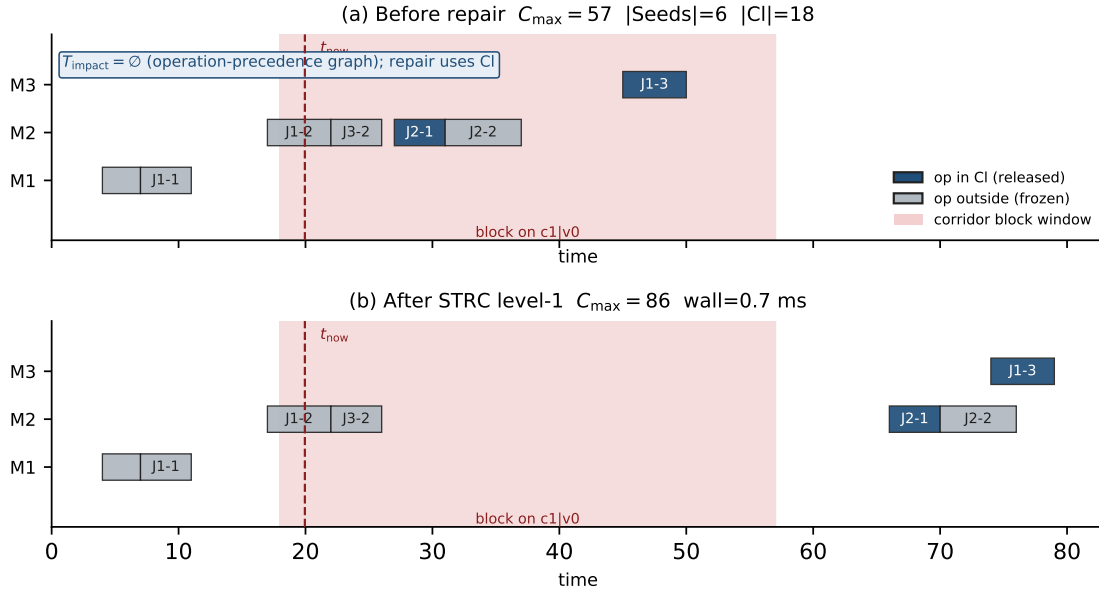


Fig. 6. Case Gantt chart (example_3x3x2, seed 42). (a) The original schedule before the blockage, coloured by membership in CI; (b) after the level-1 repair. The impact region on the operation-precedence graph is empty; the blue operations are those whose transports inside the impact set are rerouted. The figure is an *existence* demonstration: it shows what kind of local modification the statistical conclusions correspond to, and the change in C_{max} is not taken as a performance reading.

rerouted), the grey operations lie outside it (pre-committed, keeping the original plan); the red dashed line is t_{now} and the light red band is the blockage window. Figure 6(b) is the state after the repair: the time axis of the operations inside the impact set is rearranged, the schedule regains feasibility under the blockage, $C_{\text{max}} : 57 \rightarrow 86$, with a sub-millisecond runtime. The figure only shows what kind of local modification the statistical conclusions correspond to, and is not used to compare makespans; the trade-off in makespan against global rescheduling appears in E5 and the scale sweep below.

6.7 E4: structure (exploratory)

Under a blockage of an LU cut edge, the impact-set shares of funnel and high fluctuate across random seeds (Table 8). In direction funnel is indeed higher—median 0.254 against 0.204, mean 0.244 against 0.229—but *the two sets of values overlap heavily*: funnel falls in $[0.180, 0.309]$ and high in $[0.156, 0.372]$, the latter interval containing the former entirely, and a single random seed can reverse the order (in the 2024 column high's 0.372 is higher than funnel's 0.296). With five random seeds and two layouts, this difference cannot support any conclusion. The $\text{CI}/|\text{R}|$ obtained in Section 6.2 on three equal-size layouts under the busy-corridor blockage protocol likewise falls in the narrow band 0.56–0.59 and equally fails to distinguish the layouts.

Only with a batch of layouts not designed by us can the cause of this negative result be separated. The three equal-size layouts belong to one dumbbell family and differ little in geometry (see Table 9: 9–11 nodes, 8–12 corridors, and an LU cut of only 1 or 2); on them “the measure is too coarse” and “the instance set does not differ enough geometrically” are confounded and cannot be told apart. The second batch of Section 6.1 separates the two (Table 9).

The direction of the result is the opposite of what was expected, and that is precisely the point. As declared in Section 6.1, this batch has only 2 independent samples along network geometry, so it *cannot* be used to argue that “the impact-set share varies with network geometry”; what it can argue is something else. On *one and the same* 4×4 network, changing only the number and the placement of the machines from 4 to 5 moves the median impact-set share from 0.512 down to 0.305, a spread of 0.207; on one and the same 5×5 network, three machine configurations (6, 7 and 8 machines) give a spread of 0.075. For comparison, the three *parameter-matched* self-designed layouts—which are three *different* networks—fall only in 0.446–0.497, a spread of 0.051. That is: **with the network fixed and the machine configuration varying, the impact-set share can move further than it does across three different networks**—about fourfold on the 4×4 network and about one and a half times on the 5×5 one. Two qualifications are needed on the numbers of factors varied on each side: the left-hand side changes the number and the placement of machines at once, whereas the three self-designed layouts on the right are all 4-machine and change only the network. It must further be said that the right-hand baseline is weak: as noted at the start of this section, the three self-designed layouts differ little in geometry to begin with, so 0.051 is a *lower bound* on the magnitude of “changing the network”, not a representative value. Taken together, this inequality supports only “the machine-configuration dimension cannot be ignored” and does *not* support “the effect of machine configuration exceeds that of network geometry”. What holds on both networks is the direction of the inequality; the multiples themselves rest on two samples and should not be read closely.

Hence a minimum cut determined by the network alone does not explain the size of the impact set. On these 5 external instances, **the load/unload minimum cut (the LU cut) is constantly 2 and the funnel share constantly 0**—both networks place the load/unload station at a grid corner, and a corner of a 4-adjacent grid has exactly two outgoing edges, so this cut is pinned down; the family shipped with the same toolset (excluded from this batch because it permits diagonal movement) places the station at the midpoint of a grid edge, with three outgoing edges. **That is, on real layouts this measure takes only a handful of values, all fixed by the grid position of the load/unload station**, and does not vary continuously with geometry. What matters is that **these two measures do not move at all across the whole variation described above**, while the quantity to be explained has already moved by 0.207. A quantity with a constant value cannot explain one that varies this much, so it is not even a matter of “weak correlation” but of *having nothing to say*; a third measure (corridors per node) takes only two values over the whole batch (23/16 and 38/25) and is likewise insufficient to explain the within-group variation.

The reading can accordingly be tightened to: **on these 5 external instances and three self-designed layouts, the static cuts do not explain the size of the impact set.** This family of measures was designed around the tunable load/unload throat of the dumbbell layout, and once carried over to real layouts it degenerates into a constant with no explanatory power. The alternative direction therefore points at the demand side: what determines the size of the impact set is not how many paths the network *can* provide but *where the demand falls*; concrete candidate quantities are given in Section 7.2. As far as the contributions are concerned, this section is an *exploratory* negative result and supports none of them—it says only that the family of static measures we designed around the dumbbell layout should not be extrapolated, and it offers no predictive measure in their place.

6.8 E5: the trade-off against global rescheduling

This subsection originally set out to find a *crossover point*: bounded repair ahead when the budget is tight, and warm-started global rescheduling overtaking it on $C_{\max}$ once the budget is relaxed. **No such crossover exists within the 0.2–2 s range measured, and the absence extrapolates to larger budgets**, the reason being not an insufficient

Table 8. E4, the impact-set share $CI/|R|$ (blockage of an LU cut edge; random seeds 42, 7, 2024, 99, 123). The instances of this table have 12 jobs / 8 machines / 12 vehicles and a transport-to-processing time ratio calibrated to 4.0. **This convention differs from that of Table 9** (which uses 8 jobs / 4 machines / 4 vehicles, a ratio of 1.0, and ten random seeds); although the two tables share the layout names funnel and high, *the values must not be compared across tables*; only within Table 9 is the comparison like for like.

Layout	42	7	2024	99	123	Median
funnel	0.254	0.181	0.296	0.309	0.180	0.254
high	0.156	0.204	0.372	0.229	0.184	0.204

Table 9. E4, comparison with the provenance of the layouts externalized (blockage of an LU cut edge, ten random seeds; $CI/|R|$ as a median). The whole table is matched parameter for parameter and differs only in the network and the machine configuration. **The external instances live on only 2 networks** (N_1 : 4×4 with one edge removed; N_2 : 5×5 with two edges removed), and the rows within a group *share one network*, differing only in the number and the placement of the machines; the “spread” column is given *within a group*, and since group N_1 has only 2 rows and the self-designed group 3, ratios of spreads between groups are only an order-of-magnitude reference. The LU minimum cut and the funnel share *take constant values* throughout the external batch. The edge weights are the uniform values supplied by this paper, so this table is not compared against reference values from the external literature.

Source/network	Instance	Machines	Nodes/corridors	LU cut	Funnel share	$CI/ R $	Within-group spread
External N_1	LyuL2	4	16/23	2	0	0.512	0.207
External N_1	LyuL3	5	16/23	2	0	0.305	
External N_2	LyuL4	6	25/38	2	0	0.385	0.075
External N_2	LyuL5	7	25/38	2	0	0.422	
External N_2	LyuL6	8	25/38	2	0	0.460	
Self-designed	high	4	10/10	2	0.286	0.497	0.051
Self-designed	funnel	4	9/8	1	0.286	0.491	
Self-designed	mid	4	11/12	2	0.286	0.446	

sample but the shape of the two curves: R2 is a one-shot fixed point, the three budget levels within a cell sharing one and the same computation, so its $C_{\max}$ does not vary with the budget; R0+ reports the best over generations, and once the random seed is fixed a larger budget merely lets the same search trajectory run a few more generations, so its $C_{\max}$ can only decrease. Once the order of the two is settled at the *smallest* budget level, adding budget cannot reverse it, so only the smallest level needs to be examined: on the 12 instance–seed cells, R0+ is already strictly better on 11 cells, ties on 1, and is worse on none.

The *direction* of this non-existence must be made explicit: the monotonicity above extrapolates only to *larger* budgets, and smaller budgets were not swept. The same monotonicity indicates that the order should reverse on the low-budget side—R0+ pays at least one decoding per candidate evaluated (4.16 ms in the same batch, Section 6.9), so given its population size the wall clock needed merely to evaluate the initial population is already of the same order as the smallest budget level, whereas R2 needs only 2.74 ms in the same batch and is feasible on every cell. As the budget is tightened further, then, R0+ will fail to deliver even one generation, and we make no assertion about the order on that side; the smallest level measured sits right at this boundary. This does not affect the direction of our argument—what we are ruling out is “bounded repair is better on $C_{\max}$ ”, and a reversal on the low-budget side would favour bounded repair, so we do not take that reading here.

Table 10 and Figure 7 give the self-designed-layout half: two instances $\times$ random seeds $\{42, 7, 2024\} \times$ budgets $\{0.2, 1, 2\}$ s. Section 6.12 re-runs the same protocol on two *external* layouts with the same code and the same parameters;

Table 10. E5, against warm-started global rescheduling that does not rewrite finished occupancies (means over 3 random seeds). The “rewrite fraction” is the share of rewritten occupancies in the whole table. This table contains self-designed layouts only, 2 instances $\times$ 3 random seeds $\times$ 3 budget points, i.e. 6 independent cells; comparing the two $C_{\max}$ columns, R0+ is better at every budget point. Read together with the external-layout batch (Section 6.12) this becomes 12 cells and 36 points, with a point-by-point win/tie/loss of 35/1/0. R2 does not vary with the budget, so the three rows of each instance share one R2 fixed point.

Instance	Budget/s	R0+ $C_{\max}$	R2 $C_{\max}$	Rewrite fraction R0+ / R2
congested_8x4x4	0.2	279.3	299.0	0.61 / 0.57
	1.0	276.0	299.0	0.61 / 0.57
	2.0	275.7	299.0	0.61 / 0.57
example_3x3x2	0.2	78.3	88.7	0.60 / 0.60
	1.0	78.3	88.7	0.60 / 0.60
	2.0	78.3	88.7	0.60 / 0.60

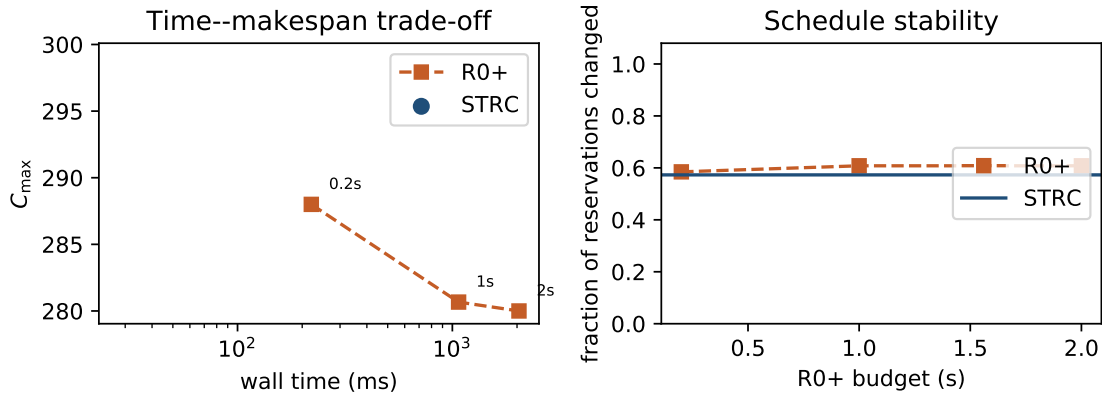


Fig. 7. E5: the budget sweep of R0+ without rewriting finished occupancies, against the fixed point of bounded repair (congested_8x4x4, means over 3 random seeds). Left: algorithm runtime against $C_{\max}$; the two curves do not intersect. Right: the rewrite fraction of occupancies, with both methods around six tenths.

read together, the two batches cover 4 instances, 12 instance–seed cells and 36 budget points, with an R0+ win/tie/loss of 35/1/0 point by point, and the monotonicity and the fixed point above were checked cell by cell on all 12 cells. The limits still have to be stated: 36 points are *not* 36 independent pieces of evidence, since the instance dimension has only 4 members, and this section therefore asserts no significance. The non-existence within and above the range measured follows from the monotonicity above and is independent of the sample size; generality along the instance dimension, however, extends only to these 12 cells—whether another batch of instances would still show no crossover is not tested here.

The makespan of bounded repair is worse over the whole range, while its algorithm runtime is two to three orders of magnitude lower (taking the 3.2 ms of level-1 rerouting against a budget of 0.2–2 s; see Figure 7, left). This runtime difference comes from a single rerouting pass without restarting the search and is *not* to the credit of the boundary drawn on the impact set— Section 6.9 establishes this with the equally fast RA comparison. The rewrite fraction, meanwhile, is already at the same 50%–66% order as R0+’s (for fewer rewrites see the RA comparison of Section 6.9: the two fractions here are already close, so stability cannot be read off this table). When the response must be at the order

of milliseconds, bounded repair can restore feasibility; when a stoppage of seconds is permitted, warm-started global rescheduling without rewriting finished occupancies is still ahead on makespan, though by a far smaller margin than the variant that reschedules from $t = 0$. $C_{\max}$ is the primary measure in this field and is therefore what we report. The comparison mentioned in Section 5.3 means that the two are each usable under a constraint on *algorithm runtime*, not that there is a budget range in which the $C_{\max}$ of R2 is better.

The R0+ of this table uses the fixed-prefix decoding of Section 5.3 and does not rewrite occupancies that have already finished. In the audit of “finished occupancies may not be rewritten”, the violations are 0 at each of the 9 budget points of the small example and of the congested example; where a downstream operation of the same job had already dispatched a vehicle while an upstream one still needed retiming, forcing the original vehicle and replanning only the tail after t_{now} preserves the historical empty prefix. Bounded repair remains uniformly zero on the same cells. Compared with the older “reschedule from $t = 0$ ” comparison, the gap in $C_{\max}$ has narrowed markedly and the rewrite fraction is no longer one hundred per cent against six tenths; but the order of the two is unchanged and there is still no crossover. What used to be mixed into the gap—“rewriting occupancies that have already finished”—has been removed, and what remains is the margin of global reassignment and resequencing over a single rerouting pass. As far as the contributions are concerned, this section establishes one *qualification* only: the net effect of the reservation impact set does not lie in makespan, and its place of application is the millisecond band; what the net effect actually is is answered by the RA comparison of Section 6.9.

6.9 E7: comparisons within the millisecond band

The comparison of the previous section is a warm-started population search with a budget of 0.2–2 s. Using it alone leaves one question open: *how much of that “two to three orders of magnitude” reading comes from the reservation impact set, and how much merely from the comparison taking longer?* This section fills in the two intermediate rungs of the ladder, the faster global recomputations.

- **RS (global right shift).** Paths, assignments and sequence are all unchanged, and the unfinished occupancies are pushed back as a whole by a delay Δ , chosen just large enough for every movable occupancy on the obstructed corridor to land after the blockage window. Occupancies already finished are not rewritten ($t_{\text{end}} \leq t_{\text{now}}$ is immovable), as in R2; because the shift is uniform, every gap either stays the same or grows, and since all constraints are of “ $\geq$ ” type, a feasible solution is obtained without any rerouting. It is one extreme of “*drawing no boundary on the impact set*”: even rerouting is given up (Section 2.1).
- **RD (re-decoding of the original chromosome).** The machine assignment and the sequence are kept, and the same fixed-prefix decoding as R0+ is run on a routing layer loaded with the blockage. It is *the faster global recomputation* in this implementation; it does not change the chromosome, so it is faster than R0+ and has less freedom to reassign and resequence.
- **RA (no restriction of the scope by the impact set, but rerouting retained).** Every occupancy *unfinished* after t_{now} is brought into the rewritable set, and then *exactly the same* rerouting replay as R2 is run. It is the other extreme of “*drawing no boundary on the impact set*”: unlike RS it retains rerouting, and therefore differs from R2 only in which occupancies may be rewritten. The two share one rerouting procedure, neither touches occupancies already finished, and both use the same blockage. It is therefore the *only* comparison in this paper that isolates the reservation impact set on its own—the difference between the two methods contains no contribution from the

Table 11. E7, comparisons within the millisecond band (50 pairs, protocol as in E3: single-window blockage on a busy corridor, $t_{\text{now}} = 0.35 C_{\text{max}}^0$, no escalation for R2). All four methods are feasible on 50/50. “Finished violated” counts the cells on which a finished occupancy was rewritten; “rewritable” is the share, among the occupancies unfinished after t_{now} , that the method brings into the rewritable set; the last column is that method’s C_{max} record against R2. RA and R2 differ only in which occupancies may be rewritten, so the difference between those two rows is entirely attributable to the reservation impact set. This table *must not* be read alongside Table 10, which covers two instances only.

Method	Finished violated	ms (median)	Mean degradation	Rewrite fraction	Rewritable	vs R2 win/tie/loss
RS global right shift	0/50	1.42	58.8%	0.627	—	7/16/27
RD re-decoding	0/50	4.16	49.0%	0.618	—	22/14/14
RA no impact boundary	0/50	2.76	49.9%	0.604	1.00	0/46/4
R2 bounded repair	0/50	2.74	49.8%	0.540	0.92	—

rerouting procedure, the protocol or the implementation. RA is already the maximal rewritable set and needs no escalation after a failure.

E3 compares two *different* objects for the boundary to be drawn on (the operation-precedence graph vs the reservation impact set), whereas RA asks: *should the scope be restricted by the impact set at all?*

Table 11 gives four readings.

First, among the methods that return a feasible plan, the only one faster than bounded repair is the global right shift, and it is worse both in makespan and in rewrite fraction. (R1, with the boundary drawn on the operation-precedence graph, is faster still, but it restores feasibility on 0/50 and is no candidate.) RS needs only 1.42 ms, about half of R2; the price is a mean makespan degradation of 58.8% (against 49.8% for R2), a per-cell record of 27/16/7 (R2 win/tie/loss), and more rewritten occupancies rather than fewer (0.627 against 0.540)—a uniform shift changes the times of every unfinished occupancy. To make the right shift *selective*—delaying only the vehicles genuinely affected and leaving the rest alone—one would first have to know which occupancies are affected, which is exactly the scope this paper delineates. **The right shift can dispense with a boundary on the impact set precisely because it has given up choosing.**

Second, the faster global recomputation still takes longer than one bounded repair. The median runtime of RD is 4.16 ms, 1.5 times that of R2. This reading is independent of the search configuration: in any decoding-based search, evaluating a candidate costs at least one decoding. The “two to three orders of magnitude” of the previous section is therefore *not* an artefact of choosing a slower comparison—replacing the comparison by the faster rung of the same implementation, bounded repair is still faster.

Third, once finished occupancies are no longer rewritten, the makespans of RD and R2 are indistinguishable on this sample, but RD rewrites more. Over the whole table, R2’s record against it is 14/14/22 (win/tie/loss), with mean degradations of 49.8% against 49.0%. The difference does not survive testing: the mean paired difference is +0.006 with a bootstrap 95% interval $[-0.007, 0.019]$ straddling 0; the two-sided Wilcoxon signed-rank test gives $p = 0.12$ (36 cells with a non-zero difference); and the mean C_{max} ratio is 0.997 with an interval $[0.988, 1.007]$, which straddles 1 as well. We therefore report only “no difference observed” and give no direction. Violations on finished occupancies already stand at 0/50: for a downgraded downstream operation the original vehicle is now forced and only the tail after the decision time is replanned, so the historical empty prefix is no longer lost. Even where RD is slightly ahead on the per-cell count, therefore, this must not be written up as “coming from rewriting occupancies that have already finished”—it is the margin of global rerouting over a single rerouting pass, with the same chromosome and

only the future scheduled. RD's rewrite fraction is 0.618, higher than R2's 0.540: fewer rewrites remain on the side that draws the boundary on the impact set.

Fourth, fewer rewrites can be read only from R2 against RA, and what is read is stability alone. RA and R2 differ only in which occupancies may be rewritten, so this pair of readings answers directly what drawing the boundary on the impact set buys in net terms. On average the reservation impact set leaves out only $1 - 0.92$ of the unfinished occupancies relative to the trivial upper bound (per cell 0.80–1.00; on the 6 cells attaining 1.00 the impact set is the set of all unfinished occupancies and the two methods coincide). It is precisely this difference of less than a tenth that brings the rewrite fraction down from 0.604 to 0.540: a mean reduction of 0.064 and a median reduction of 0.029, per cell 31/18/1 (R2 more stable/tie/less stable), two-sided Wilcoxon signed-rank $p = 8.8 \times 10^{-7}$. The effective sample size must be stated: the 6 coincident cells are not comparisons between the two methods; removing them leaves 44 cells, a per-cell record of 31/12/1 and a mean reduction of 0.073 (the number of cells with a non-zero difference for Wilcoxon is 32 under either accounting); removal makes the estimated reduction *larger*, so reporting over 50 cells is the more conservative side. The direction is highly consistent and the *magnitude is small*—on the single counterexample cell R2 in fact rewrites 0.007 more, which is recorded as well. Meanwhile the algorithm runtimes are almost identical (2.74 against 2.76 ms) and so are the makespans (mean ratio 0.999, per-cell R2 win/tie/loss 4/46/0, never worse, the most favourable cell being 0.970).

These three items together are the point. **The identical runtimes show that the millisecond-level response comes from a single rerouting pass without restarting the search, and not from the reservation impact set (readings in Table 11 of this section; a single rerouting pass is itself existing practice, see Table 2). The identical makespans show that the impact set is not an optimization device. The markedly lower rewrite fraction shows that drawing the boundary delivers exactly the quantity promised by Proposition 3: the corridor, interval and vehicle of the occupancies outside the set are unchanged.** The role of the impact set is to *mark which occupancies need not be changed*. The magnitude grows as the impact set excludes more, the direction is consistent, but the excluded set alone accounts for only part of it—even counting every occupancy forbidden to be rewritten, that does not make up the whole $0.604 - 0.540$ gap, and the remainder comes from differences in the rerouting outcome itself once the modifiable set shrinks; we do not decompose it further: on an instance where the impact set is close to all unfinished occupancies (congested_8x4x4, rewritable share 0.94) the reduction is only 0.010, whereas on an instance where more is excluded (S8x4x4_mid, 0.91) it reaches 0.118. Two instances cannot fix a proportionality, and all that can be said is the direction; the per-cell distribution follows.

Let us write this direction up as a post hoc rule. Figure 8 plots the 50 pairs with the rewritable share $Cl/|R^o|$ on the horizontal axis and the reduction in rewrite fraction of R2 relative to RA on the vertical axis. Taking $Cl/|R^o| \geq 0.93$ on this sample: above that line, 23 cells have a mean reduction of 0.029, and below it, 27 cells have a mean reduction of 0.094. The reductions have all fallen to zero once the share rises above 0.97, but for two different reasons: on the 6 cells whose share is exactly 1.00 the impact set is the set of all unfinished occupancies and the two methods *coincide*; on the remaining cells the share is below 1.00 and the reduction is nevertheless 0, which is measured rather than identical by construction. This is a post hoc stratification whose threshold is chosen on this table, and no extrapolated theorem is claimed; it answers “when is drawing the boundary on the impact set still worthwhile”, not the structural prediction of the impact-set size (which remains unestablished, Section 6.7).

Reading the four methods together closes the matter as follows: under the two constraints “do not rewrite occupancies that have already finished” and “respond at the order of milliseconds”, **the net benefit of drawing the boundary on the impact set shows up in the amount of rewriting alone—at equal speed and equal makespan, not drawing**

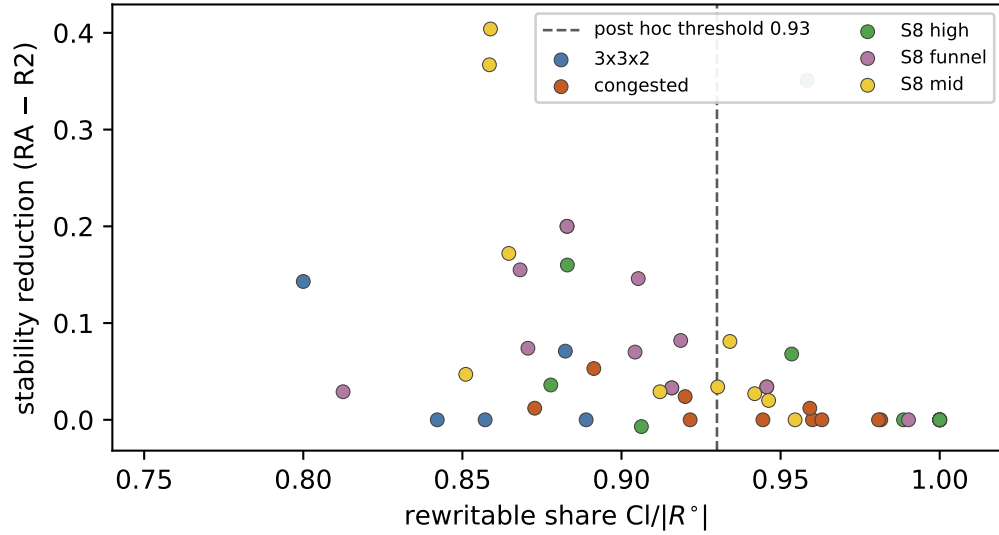


Fig. 8. E7 post hoc rule: rewritable share against stability reduction (50 pairs, R2 against RA). The dashed line is the in-sample threshold 0.93; a positive value on the vertical axis means a lower rewrite fraction once the boundary is drawn on the impact set.

it means rewriting about 6 percentage points more occupancies (about a twelfth more in relative terms), and it includes *no* advantage in speed or in makespan. This layer answers the net benefit of drawing the boundary; “whether the object the boundary is drawn on stands up” is supported by Sections 6.2–6.4. Once the runtime constraint is relaxed, the shorter makespan still comes from the warm-started search (Section 6.8), but that is no longer the millisecond band.

6.10 A sweep over the disturbance scale

Table 12 and Figure 9 report the φ sweep (congested_8x4x4, 10 random seeds; R0+ budget 2 s). The per-level readings are in the table; $C_{\max}$ is still better for R0+ throughout, in agreement with Section 6.8. On the small example example_3x3x2 every cell is likewise feasible (10 random seeds $\times$ five values of φ), with bounded repair at the order of 1 ms and a runtime ratio of $87 \times - 3051 \times$ (per-cell extremes across the two instances). Here the denominator is the measured wall clock of bounded repair while the numerator is the *budget* of R0+, and R0+ is better on $C_{\max}$, so this is not a “speed-up” and constitutes no comparison at equal quality (Section 6.1 has already declared that the methods are not compared at a common budget level).

Bounded repair here has scope escalation switched on, so this section is not directly comparable with E3/E5: escalation is the step that pulls the feasibility rate to 100% and has precedents of its own (Section 2.1). This section *supports none of the contributions* and establishes one qualification only: even when the disturbance fraction rises to $\varphi = 1$ (every future operation disturbed), the algorithm runtime stays at the order of tens of milliseconds and does not collapse with size; and this runtime comes from the single rerouting pass and is *not* to the credit of the boundary drawn on the impact set (Section 6.9).

Table 12. Comparison across the disturbance scale (congested_8x4x4, averaged over 10 random seeds). Bounded repair includes escalation; the R0+ budget is 2 s. Dispersion is reported as an interquartile range: the spread of the STRC runtime over the whole sweep is 2.2–23.8 ms, and the per-level IQR of the R0+ $C_{\max}$ lies between 8 and 76. **At the two smallest levels the within-level IQR of the “runtime ratio” column already exceeds the between-level movement of that column, so it is to be read as an order of magnitude only and does not indicate a trend in φ .** That column is the per-level mean of the *per-cell* ratios (pair first, then aggregate), and therefore does not equal the ratio of the two neighbouring column means—the two necessarily differ by the convexity of $1/x$, so please use the per-cell data when recomputing. The numerator is the wall-clock *budget* of R0+ rather than its measured runtime, so this is not a comparison at equal quality (Section 6.1).

φ	STRC feasible	STRC ms	R0+ $C_{\max}$	Runtime ratio
0.10	100%	9.1	103.5	406×
0.25	100%	11.1	107.0	325×
0.50	100%	8.6	111.8	333×
0.75	100%	8.4	125.8	254×
1.00	100%	10.7	148.1	203×

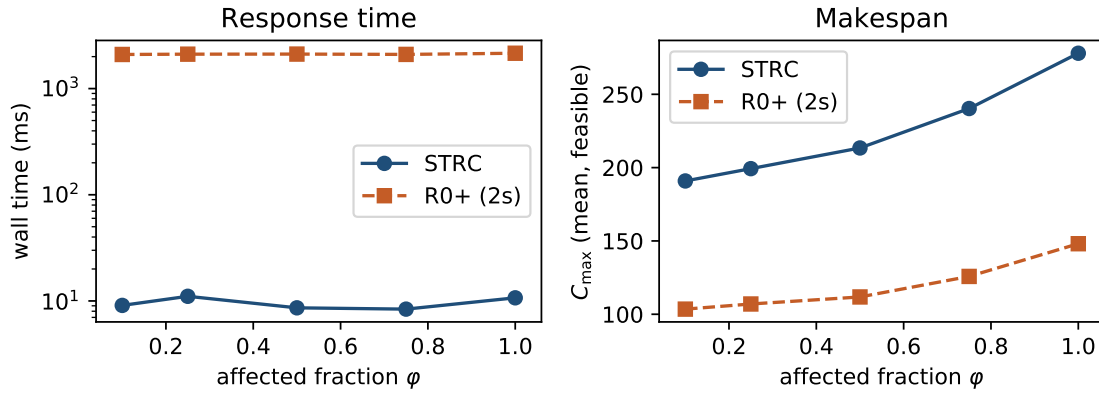


Fig. 9. Sweep over the disturbance scale (congested_8x4x4, 10 random seeds): the algorithm runtime (logarithmic axis) and the makespan of bounded repair and of R0+ as functions of φ . The curves are per-level means, and the dispersion convention is that of Table 12 (interquartile range); on the runtime side the spread over the whole sweep is 2.2–23.8 ms, always at the order of tens of milliseconds, and this is the only reading on which this section rests its argument.

6.11 E8: extrapolation in instance size

The previous section swept the scale of the *disturbance*, with the instance fixed at 8 jobs and 4 machines. This section changes dimension: the disturbance protocol is fixed and the instance itself is scaled up. The crux of this question is not runtime—that it does not collapse is to be expected—but **whether the impact-set share rises towards 1 with size, so that “bounded” becomes an empty word.**

The size sequence takes $2k$ jobs, k machines and k vehicles for $k = 4, \dots, 10$, that is from $8 \times 4 \times 4$ to $20 \times 10 \times 10$, giving 7 levels $\times$ 10 random seeds = 70 pairs. The congestion level, the heterogeneity, the flexibility, the transport-to-processing time ratio, the load/unload minimum cut and the remote cut are all calibrated to the same targets, and *only the size varies*. The three comparisons are defined as in Section 6.9.

The per-level readings are in Table 13.

Table 13. E8, sweep over instance size (7 levels $\times$ 10 random seeds). “Impact set” is the share of the reservation impact set in the *whole table*; the attainable upper bound of that ratio is $|R^\circ|/|R|$, whose mean in this batch is about 0.61; with the rewritable set as denominator it is 0.78–0.91 (see the body). Runtimes are medians, and the “RD/R2” column is the ratio of those two *medians*; the monotonicity discussed in the body uses per-level means of the per-cell pairs, and the two conventions move alike between levels but differ in value. The last column is R2’s $C_{\max}$ record against RS.

Size	$ R $	Impact set	R2 ms	RD ms	RD/R2	R2 vs RS
$8 \times 4 \times 4$	148	0.561	2.9	4.4	1.5 $\times$	7/1/2
$10 \times 5 \times 5$	172	0.485	3.2	6.1	1.9 $\times$	9/0/1
$12 \times 6 \times 6$	230	0.481	4.5	10.7	2.4 $\times$	7/1/2
$14 \times 7 \times 7$	281	0.492	6.1	14.5	2.4 $\times$	8/0/2
$16 \times 8 \times 8$	291	0.534	6.4	16.4	2.6 $\times$	10/0/0
$18 \times 9 \times 9$	356	0.535	8.2	26.4	3.2 $\times$	10/0/0
$20 \times 10 \times 10$	402	0.476	10.4	34.2	3.3 $\times$	10/0/0

The impact-set share does not deteriorate with size, but the worry about “approaching 1” is only answered once the denominator is changed. The seven levels of $Cl/|R|$ fluctuate between 0.48 and 0.56 and are not monotone (the first level is the highest and the last the lowest, while the fifth and sixth come back above 0.53). The denominator must be made explicit here, or too strong a conclusion will be read off: Cl takes only the occupancies unfinished after t_{now} , whereas the mean $|R^\circ|/|R|$ in this batch is only 0.61, so the attainable upper bound of $Cl/|R|$ is not 1 but about 0.61—using it to answer the question above is beside the point. With the rewritable set as denominator, the seven levels of $Cl/|R^\circ|$ fall in 0.78–0.91 (per cell up to 1.00), of the same order as the 92% of the main batch in Sections 6.5 and 6.9. What this section supports is therefore that *the share does not deteriorate with size*, and *not* that “bounded” is generous in absolute terms—the latter has already been denied in Remark 2. Nor is this an asymptotic claim: seven levels are insufficient to fit any growth rate, and all that can be said is that over this range it has not degraded.

The algorithm runtime grows faster than the total number of occupancies. As $|R|$ grows by a factor of 2.7, R2’s median runtime grows from 2.9 to 10.4 ms, a factor of 3.6, and remains at the order of tens of milliseconds; seven levels fix no exponent, so only the ratio of the two endpoints is reported, and it is written neither as “linear” nor as “superlinear”. The number of cells feasible with level-1 rerouting is 69/70; the single failing cell ($10 \times 5 \times 5$, seed 42) is caught by scope escalation in round 3, with a total runtime of 12.0 ms and the same makespan as the level-1 attempt, so the final feasibility is 70/70.

The gap between the two comparisons widens with size. The runtime ratio of the faster global recomputation to bounded repair rises from 1.5 $\times$ to 3.3 $\times$ —the larger the instance, the less a full recomputation pays. Only the two endpoints are reported here: the per-level ratio is *not monotone* (the per-level mean of the per-cell pairs dips at $k = 6 \rightarrow 7$). The two endpoints differ by 1.8, whereas the interquartile range within any level does not exceed 0.74 and the sampling fluctuation of a per-level mean over 10 cells is far smaller than that span, so *the overall rise* is not noise; the shape of the growth, however, we do not report. Against the right shift, which draws no boundary on the impact set, R2’s $C_{\max}$ record is 10/0/0 at each of the last three levels, while at the first four levels it varies between 7 and 9 wins with no monotone pattern; each level has only 10 cells, so only the clean sweeps of the last three levels are reported and no trend is claimed.

Limits of the convention. This size sequence has one congestion level and one instance seed (the instance at each level is itself generated with seed 42), so what it tests is the *size* dimension and it cannot replace the comparison over congestion structures in the main batch of Section 6.1; nor were the sweeps of Sections 6.5 and 6.10 re-run on this batch.

As far as the contributions are concerned, this section adds one qualification in the *size direction* to the boundedness of C2: the share is not diluted over these seven levels, but this bears on no net benefit and constitutes no asymptotic conclusion.

6.12 Reproduction on the external-layout batch

Once the second batch of Section 6.1 replaces the provenance of the layouts by an external source, the E1/E2/E3/E5 readings on the 50 instance–seed pairs agree *item by item in direction* with those of the main batch: the operation-precedence graph returns an empty scope on 50/50, structural containment (that is, zero leakage) holds on 50/50, and the fields outside the set are unchanged entry by entry on 50/50; under the boundary ablation the number of cells feasible with level-1 rerouting is 0/50, while with the boundary drawn on the reservation impact set it is 50/50. Under the E1 protocol, the per-layout median impact-set share falls in 0.438–0.547; the comparable reading in the main batch is the median 0.571 over the 50 pooled pairs (0.56–0.59 is the spread of the per-instance *means*, a different convention that should not be read directly against the preceding clause). The external batch is slightly lower overall, but both are of the order of about a half, and the shortfall is of the same order as the between-instance variation (see the caption of the figure in Section 6.2); we offer no attribution for it. The trade-off against global rescheduling is unchanged as well: R0+ has the better $C_{\max}$ at 17/18 of the budget points, with the remaining 1 point a tie (LyuL6_8m, seed 2024, budget 0.2 s) and none worse. This batch runs the same E5 code with the same parameters as the main batch, which is why Section 6.8 reads the two batches together over 12 instance–seed cells; the tied cell also satisfies the monotonicity argument of that section, and the crossover still does not exist.

The value of this batch lies not in new performance numbers but in answering “whether these phenomena hold only on layouts we designed ourselves”: the three readings above hold equally on a batch of topologies not chosen by us, so the object layer of C1 and C2 does not depend on our layout design. The only conclusion this batch revises is structural predictability (Section 6.7), and that was a negative result to begin with.

6.13 Summary

The readings of this chapter are grouped by the contribution layer they support; the limitations of each, and which layer each does and does not threaten, are collected in Section 7.1.

The object layer (C1, C2: on what the boundary is drawn, and whether the set so drawn is uniquely determined) is supported by four groups of readings. Under corridor blockage the scope on the operation-precedence graph is empty while the original schedule is indeed infeasible (E1, 50/50 on 50 pairs); the reservation impact set is closed under the dependency edges and the fields outside it are unchanged entry by entry (E2, 50/50 and 50/50); replacing only the object the boundary is drawn on under one and the same rerouting procedure moves the rate at which feasibility is restored from 0/50 to 50/50 (E3), so that feasibility comes from the object the boundary is drawn on and not from the quality of the rerouting procedure; and in the cross of four disturbance types with two ways of drawing the boundary the A/B dichotomy holds without exception, while on Class A the reservation impact set uniformly contains the operation-precedence boundary (E6). Propositions 1–3 give the corresponding structural statements, of which Proposition 3 remains a sufficient condition only.

The net-benefit layer (C3: what drawing the boundary this way buys) is supported by one group of readings only: switching to the comparison that does not restrict the scope by the impact set (RA, with the same rerouting as R2 and likewise never touching occupancies already finished), algorithm runtime and $C_{\max}$ barely change, while the rewrite fraction rises from 0.540 to 0.604 (per cell 31/18/1, $p = 8.8 \times 10^{-7}$; Section 6.9). Hence the millisecond-level response

comes from the single rerouting pass and not from the impact set, and **fewer rewrites can be read only from this comparison**. Against warm-started global rescheduling, bounded repair is better on $C_{\max}$ on no cell, and relaxing the budget produces no crossover (E5); E7 then removes the doubt that “the comparison simply took longer”.

The remaining two groups of readings support no contribution and only add qualifications: the scale sweep has reached 20 jobs and 10 machines, and the impact-set share with the rewritable set as denominator has not deteriorated with size (E8); structural predictability is a negative result (E4). The second batch of instances (50 pairs) only tests whether the phenomena above depend on the layouts we chose ourselves, and E1, E2, E3 and E5 all reproduce item by item.

7 Conclusion

This paper makes three contributions. **First**, it identifies a class of damage beyond the reach of the operation-precedence graph: a corridor blockage changes no operation assignment, and the scope delineated by affected operations is empty while the original schedule is indeed infeasible (50/50 on 50 pairs, Section 6.2). **Second**, it defines the rescheduling scope as the reservation impact set (STRC): once the dependency relations are given, whether an occupancy belongs to the set is uniquely determined, and no dependency edge leads from inside the set to outside it (50/50, 50/50, Section 6.3); this layer does not depend on the net-benefit readings below. **Third**, it gives a local recovery matched to this boundary and keeps straight where each reading comes from: replacing only the object the boundary is drawn on under one and the same rerouting procedure moves the rate at which feasibility is restored from 0/50 to 50/50 (Section 6.4), so that feasibility comes from the object the boundary is drawn on and not from the quality of the rerouting procedure.

The several mechanisms that make up the impact set—yielding relations, rerouting inside the set while keeping the plan outside it, and enlarging the rewritable scope after a failure—have each appeared in the existing literature (Section 2.5), and we do not claim them as contributions. What this paper claims is this: the operation-precedence graph contains, by definition, no yielding relations among corridor occupancies, and therefore cannot reach a disturbance that merely invalidates occupancies; once the scope is moved onto the reservation dependency graph, the occupancies that must be rewritten are uniquely determined by the given order, rather than by a heuristic radius.

When the rewritable set is changed from the reservation impact set to “every occupancy unfinished after t_{now} ” and everything else is left untouched, algorithm runtime and makespan barely move, while the rewrite fraction rises from 0.540 to 0.604 (Section 6.9). Hence the millisecond-level response comes from a single rerouting pass without restarting the search; **fewer rewrites can be read only from this comparison, and what it delivers is the quantity promised by Proposition 3: the occupancies outside the set keep the original plan**. The magnitude of the benefit depends on how far the impact set falls short of that trivial upper bound, and so tends to vanish as the two approach each other.

7.1 Limitations

The four items below are ordered by their bearing on the readings above, and Section 7.2 gives a direction for each.

- (1) **The makespan is worse than that of warm-started global rescheduling without rewriting finished occupancies, and relaxing the budget produces no crossover**. Bounded repair trades makespan for algorithm runtime, and this disadvantage does not reverse as the budget is relaxed on the 12 instance–seed cells measured (Section 6.8). This item **threatens** the layer of “the net benefit of drawing the boundary on the reservation impact set” and **does not threaten** the layer of “which occupancies must be rewritten is uniquely determined by the

structure”: fewer rewrites can be read only from the comparison with RA and not from the comparison with R0+ (Section 6.9).

- (2) **For the reservation impact set, the guarantee of Proposition 3 is only a sufficient condition, the dependency edge set has not been tested exhaustively and independently, the rewritable set is not tight, and its size cannot be predicted by static measures that depend on the network alone.** The impact set covers about 92% of the occupancies unfinished after t_{now} , and we make no minimality claim (Remark 2); the magnitude of the net benefit depends on how far the impact set falls short of that trivial upper bound, so the benefit tends to vanish as the two approach each other. The family of static cuts has already been ruled out in Section 6.7, but that negative result bears only on “the static cuts can explain the size of the impact set” and does not amount to “network geometry is unrelated to the size of the impact set”. This item **threatens** the strength and the tightness of the guarantees at the object layer and **does not threaten** the diagnostic reading that “drawing the boundary on the operation-precedence graph returns an empty scope”.
- (3) **The implementation covers less than the problem range described in the body: Class-B disturbances are answered by rerouting alone, Class A additionally opens vehicle swap and reassignment but not a change of operation order, and by Assumption A3 only one response is made.** Opening up the operation order degenerates towards global rescheduling, and “bounded” thereby loses its boundary; disturbance streams, prediction error and rolling response are all untouched. The size dimension has been swept to 20 jobs and 10 machines with no rise in the impact-set share, but the number of levels swept cannot support any asymptotic statement. This item **threatens** extrapolation of the readings to larger sizes and to continuous disturbances and **does not threaten** the readings already reported within the protocol.
- (4) **An absent external reference: this class of study has no public benchmark that could be substituted in, the provenance of the instances is externalized only to the level of topology, and the readings have not been reproduced on a second implementation.** The public instance families [3, 6] publish transport times as a matrix of machine-to-machine travel times rather than as a network, and of the 11 layouts the two families provide between them, checked one by one, not one could be recovered as an undirected corridor graph consistent with the published matrix, so R , G_R , $Cl(\cdot)$ and Class-B disturbances cannot be defined on them at all (Section 6.1). The second batch of instances is constructed from the appendix layouts of Lyu et al. [10], so the layouts are no longer of our design, but the per-leg travel times and the job data still come from us, so they cannot be used to reproduce the reference values of Lyu et al. or of van Os and Basten [20], and their coverage is narrower than that of the main batch. This item **threatens** external validity and **does not threaten** the relative readings among the comparisons within one implementation.

7.2 Future work

The four items below correspond one to one with the four limitations of the previous section.

One: facing the makespan disadvantage over the whole range (limitation 1). The rewriting of finished occupancies by the prefix decoding has already been eliminated in the main batch (0 violations at the 18 budget points of E5, and 0/50 for RD in E7). What remains of limitation 1 is C_{max} itself, and we do not evade it by switching to a differently weighted measure. Narrowing the gap would require limited reassignment or resequencing while still not rewriting finished occupancies, and that degenerates towards global rescheduling.

Two: making the guarantee for the reservation impact set more complete and tighter, and establishing a characterization of its size (limitation 2). *Completeness*: abutment has been folded into the yield edges, and after re-running, the probe leaks 0. The next step is to probe with a different form of disturbance (shortening an occupancy, say, or delaying by more than 1 unit) rather than to add the same edge again. *Tightness*, pointing at a smallest rewritable set: define minimality under a fixed repair capability and give the gap between the impact set and it—an unavoidable step in carrying “bounded” from a sufficient condition to a tight bound; since the impact set currently covers about 92% of the unfinished occupancies, that gap may be considerable. *Characterizing the size* currently rests on a single post hoc rule (the rewritable share, Figure 8), which varies with the schedule; it must be replaced by a family of measures computable before the disturbance. Section 6.7 has already ruled out the family of static cuts—it takes constant values on real layouts, whereas changing only the machine configuration on one and the same network makes it move: on the 4×4 network, going from 4 to 5 machines moves it by 0.207, and on the 5×5 network three configurations (6, 7 and 8 machines) move it by 0.075. The latter suggests the direction: what appears to matter is the *spatial distribution of demand*, not the throughput the network provides. The next candidate is the temporal density of corridor occupancies in the neighbourhood of t_{now} , for instance the number of overlapping occupancies of the disturbed corridor within its busy window, and the out-degree of that corridor in the yield graph. Such quantities vary with the schedule and are no longer purely topological, so the phrase “predictability” must be changed accordingly to predicting the size given the layout and the current schedule.

Three: relaxing Assumption A3 (limitation 3; A3 is the single, fully observed disturbance). The next step is not to open up resequencing further but to relax that assumption, and to examine whether the impact set accumulates and swells under successive disturbances, and whether an explicit “re-homing” step is needed between two repairs.

Four: externalizing the time calibration as well (limitation 4). The provenance of the layouts has settled half of this; completing the other half still requires an instance family that publishes *both an exclusive network and per-leg durations*, and a sweep over self-assigned edge weights is no substitute. As for a second independent implementation, it needs no separate project: it can be done along with any of the reproductions above.

Acknowledgments

This work was supported by <to be supplied: funding programme name and grant number>. The authors thank the anonymous reviewers for their comments on the experimental conventions and on the boundaries of the conclusions.

Conflict of interest statement

The authors declare that they have no conflict of interest related to the work reported here.

References

- [1] Riyad J. Abumaizar and Joseph A. Svestka. 1997. Rescheduling job shops under random disruptions. *International Journal of Production Research* 35, 7 (1997), 2065–2082. doi:10.1080/002075497195074
- [2] M. Selim Aktürk and Elif Görgülü. 1999. Match-up scheduling under a machine breakdown. *European Journal of Operational Research* 112, 1 (1999), 81–97. doi:10.1016/S0377-2217(97)00396-2
- [3] Ümit Bilge and Gündüz Ulusoy. 1995. A Time Window Approach to Simultaneous Scheduling of Machines and Material Handling System in an FMS. *Operations Research* 43, 6 (1995), 1058–1070. doi:10.1287/opre.43.6.1058
- [4] Andrea D’Ariano, Dario Pacciarelli, and Marco Pranzo. 2007. A branch and bound algorithm for scheduling trains in a railway network. *European Journal of Operational Research* 183, 2 (2007), 643–657. doi:10.1016/j.ejor.2006.10.034
- [5] Jie Fu, Bin Yang, Zhixing Chang, Yuanrong Zhang, Jiarui Wang, Xiaotong Wang, and Lei Wang. 2025. Integrated Production–Logistics Scheduling in Flexible Assembly Shops Using an Improved Genetic Algorithm. *Machines* 13, 12 (2025), 1090.

- [6] Seyed Mahdi Homayouni and Dalila B. M. M. Fontes. 2021. Production and Transport Scheduling in Flexible Job Shop Manufacturing Systems. *Journal of Global Optimization* 79, 2 (2021), 463–502. doi:10.1007/s10898-021-00992-6
- [7] Jeonghyeon Kim and Junwoo Kim. 2026. Congestion-Aware Scheduling for Large Fleets of AGVs Using Discrete Event Simulation. *Electronics* 15, 1 (2026), 139. doi:10.3390/electronics15010139
- [8] Jürgen Kuster, Dietmar Jannach, and Gerhard Friedrich. 2007. Local Rescheduling: A novel approach for efficient response to schedule disruptions. In *2007 IEEE Symposium on Computational Intelligence in Scheduling (CISched)*. IEEE, 79–86. doi:10.1109/SCIS.2007.367673
- [9] Ruiqi Li, Jianlin Mao, Xing Wu, Wenna Zhou, Chengze Qian, and Haoshuang Du. 2026. A New Framework for Job Shop Integrated Scheduling and Vehicle Path Planning Problem. *Sensors* 26, 2 (2026), 543. doi:10.3390/s26020543
- [10] Xiangfei Lyu, Yuchuan Song, Changzheng He, Qi Lei, and Weifei Guo. 2019. Approach to Integrated Scheduling Problems Considering Optimal Number of Automated Guided Vehicles and Conflict-Free Routing in Flexible Manufacturing Systems. *IEEE Access* 7 (2019), 74909–74924. doi:10.1109/ACCESS.2019.2919109
- [11] Nir Malka, Guy Shani, and Roni Stern. 2024. Online Planning for Multi Agent Path Finding in Inaccurate Maps. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 10214–10221. doi:10.1109/IROS58592.2024.10801975
- [12] Alessandro Mascis and Dario Pacciarelli. 2002. Job-shop scheduling with blocking and no-wait constraints. *European Journal of Operational Research* 143, 3 (2002), 498–517. doi:10.1016/S0377-2217(01)00338-1
- [13] Leilei Meng, Weiyao Cheng, Chaoyong Zhang, Kaizhou Gao, Biao Zhang, and Yaping Ren. 2025. Novel CP Models and CP-Assisted Meta-Heuristic Algorithm for Flexible Job Shop Scheduling Benchmark Problem with Multi-AGV. *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 55, 11 (2025), 8455–8468. doi:10.1109/TSMC.2025.3604355
- [14] Djamila Ouelhadj and Sanja Petrovic. 2009. A Survey of Dynamic Scheduling in Manufacturing Systems. *Journal of Scheduling* 12, 4 (2009), 417–431. doi:10.1007/s10951-008-0090-8
- [15] Haoxiang Qin, Yi Xiang, Yuyan Han, Yuting Wang, Junqing Li, and Quanke Pan. 2026. A Knowledge Region Selection Enhanced Quality-Diversity Algorithm for Real-World Flexible Job Shop Scheduling with Automated Guided Vehicles Transportation. *Engineering Applications of Artificial Intelligence* 164 (2026), 113352. doi:10.1016/j.engappai.2025.113352
- [16] Velusamy Subramaniam and Amritpal Singh Raheja. 2003. mAOR: A heuristic-based reactive repair mechanism for job shop schedules. *The International Journal of Advanced Manufacturing Technology* 22, 9-10 (2003), 669–680. doi:10.1007/s00170-003-1601-6
- [17] Jiachen Sun, Zifeng Xu, Zhenhao Yan, Lilan Liu, and Yixiang Zhang. 2023. An Approach to Integrated Scheduling of Flexible Job-Shop Considering Conflict-Free Routing Problems. *Sensors* 23, 9 (2023), 4526. doi:10.3390/s23094526
- [18] Jiří Švancara, Marek Vlk, Roni Stern, Dor Atzmon, and Roman Barták. 2019. Online Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 7732–7739. doi:10.1609/aaai.v33i01.33017732
- [19] Hongtao Tang, Xuewei Zhang, Jiawei Huang, Xuesong Xu, and Jiansha Lu. 2026. Integrated Scheduling of Multi-Objective Lot-Streaming Hybrid Flowshop with AGV under Dynamic Environments. *International Journal of Industrial Engineering Computations* 17, 2 (2026), 295–316. doi:10.5267/j.ijiec.2025.9.003
- [20] Roel W. M. van Os and Twan Basten. 2026. An Exact Decomposition-Based Approach to the Conflict-Free-Transportation-Constrained Flexible Job-Shop Scheduling Problem. *Computers & Operations Research* 187 (2026), 107342. doi:10.1016/j.cor.2025.107342
- [21] Guilherme E Vieira, Jeffrey W Herrmann, and Edward Lin. 2003. Rescheduling Manufacturing Systems: A Framework of Strategies, Policies, and Methods. *Journal of Scheduling* 6, 1 (2003), 39–62. doi:10.1023/A:1022235519958
- [22] Bin Xin, Sai Lu, Qing Wang, and Fang Deng. 2025. A Review of Flexible Job Shop Scheduling Problems Considering Transportation Vehicles. *Frontiers of Information Technology & Electronic Engineering* 26, 3 (2025), 332–353. doi:10.1631/FITEE.2300795
- [23] Yannik Zeiträg and José Rui Figueira. 2025. A novel machine-learning rolling horizon heuristic for dynamic lot-sizing and job shop scheduling problems. *International Journal of Production Research* 63, 12 (2025), 4563–4589.
- [24] Hongliang Zhang, Chaoqun Qin, Wenhui Zhang, Zhenxing Xu, Gongjie Xu, and Zhenhua Gao. 2023. Energy-Saving Scheduling for Flexible Job Shop Problem with AGV Transportation Considering Emergencies. *Systems* 11, 2 (2023), 103. doi:10.3390/systems11020103
- [25] Yulun Zhang, He Jiang, Varun Bhatt, Stefanos Nikolaidis, and Jiaoyang Li. 2024. Guidance Graph Optimization for Lifelong Multi-Agent Path Finding. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*.
- [26] Ziyang Zhao, Shixin Liu, MengChu Zhou, Xiwang Guo, and Liang Qi. 2019. Decomposition method for new single-machine scheduling problems from steel production systems. *IEEE Transactions on Automation Science and Engineering* 17, 3 (2019), 1376–1387.